ORCA 机器人仿真生态

OrcaLab · OrcaGym · SouthGrid

SouthGrid
端到端实现指南

任务二 / 任务三 · 数据采集 → 模型训练 → 推理部署 → 评分接入

覆盖数据采集、模型训练、推理部署、评分接入四个阶段的端到端实现指南：基础环境搭建（OrcaLab / Conda / OpenPI 双环境）、通用概念与约定、任务二语言指令按钮操作、任务三电力工具归纳收置、训练后模型控制 ORCA 机器人的链路架构、ScorerClient 评分系统本地与远程接入，并附 OrcaGym API 参考手册、相机取帧到 VLA 推理的完整实践与跨平台训练指南。

文档版本：基于 SouthGrid 仓库当前代码及关联评分客户端

阅读前提：Linux · Conda · Python · Git 基础

适用对象：参赛开发者（0 到 1 全流程）

出品：上海松应科技有限公司

本文档仅供参赛队伍参考，禁止扩散传播

前言

> **本文档基于提供的代码仓库及相关资料撰写。若本文档内容与代码仓库中的内容不一致，请以代码仓库中的内容为准。**

本文档为任务二（语言指令按钮操作）、任务三（电力工具归纳收置）的端到端实现指南，内容覆盖数据采集、模型训练、推理部署与评分接入四个阶段，基于 SouthGrid 仓库当前代码及关联评分客户端整理。阅读需具备 Linux / Conda / Python / Git 基础。

全文共十三章。第一至三章为基础部分：总体流程概览、基础环境搭建（OrcaLab 7.2 及以上版本安装与资产订阅、SouthGrid 仓库克隆、Conda 环境、OpenPI 独立训练环境）、通用概念与约定（两套 Python 环境、机器人形态、LeRobot v2.1 数据集格式、动作空间、相机端口）。第四、五章分别为任务二与任务三的实现流程，覆盖数据采集（智元 OmniPicker / 宇树 G1）、数据检查、OpenPI 模型训练与推理部署。第六章说明训练后模型控制 ORCA 机器人的链路架构与调试方法；第七章为评分系统接入，包括客户端安装、部署关系、本地与远程推理接入、API 速查与结果查看；第八至十章为常见问题排查、关键文件与命令速查及实施要点；第十一至十三章为进阶参考：OrcaGym API 完整参考手册、相机配置→取帧→OBS 构造→VLA 推理完整实践、独立机器人 XML 模型与跨平台训练。

说明：文中代码均为参考样例，可在不破坏评分规则的前提下自行重新实现控制、训练或评分接入逻辑；ScorerClient 及其本地评分服务必须与 ORCA 仿真运行在同一台物理机，策略服务器可远程部署并通过 SSH 隧道或直联访问；评分客户端须通过华为云 Git 仓库（SouthGrid 标签）安装，使用本地源码目录安装可能导致评分服务无法自动启动；Linux 下进行视频录制需额外安装 pyautogui、python-xlib、imageio-ffmpeg、mss 等 Python 包及 ffmpeg、xdotool、wmctrl 等系统组件，Wayland 环境还需 grim、slurp。

目　录

第 1 章　总体流程概览 7

1.1 代码样例声明 7

第 2 章　基础环境搭建 9

2.1 硬件与系统要求 9

2.2 安装 OrcaLab 7.2 及以上版本与订阅资产 9

2.3 克隆 SouthGrid 9

2.4 创建 Conda 环境 10

2.5 环境验证 10

2.6 安装 OpenPI 训练环境（独立） 11

第 3 章　通用概念与约定 12

3.1 两套 Python 环境 12

3.2 机器人形态 12

3.3 LeRobot v2.1 数据集格式 12

3.4 动作空间约定 13

3.5 相机端口约定 13

第 4 章　任务二：语言指令按钮操作 15

4.1 任务目标 15

4.2 选择技术路线 15

4.3 数据采集（智元 OmniPicker） 15

4.4 数据采集（宇树 G1） 17

4.5 数据检查 18

4.6 模型训练（OpenPI） 19

4.7 推理部署 23

第 5 章　任务三：电力工具归纳收置 25

5.1 任务目标 25

5.3 模型训练（OpenPI） 27

5.4 推理部署 27

5.5 任务三评分脚本（需自行实现） 28

第 6 章　训练后的模型如何控制 ORCA 机器人 30

6.1 整体链路 30

6.2 策略服务进程（OpenPI 侧） 30

6.3 推理客户端进程（ORCA 侧） 31

6.4 宇树 G1 的控制差异 37

6.5 关键调试信息 38

第 7 章　评分系统接入 39

7.1 评分客户端安装 39

7.2 ScorerClient 与 ORCA 的部署关系 42

7.3 本地推理评分接入 43

7.4 远程推理评分接入 44

7.5 模型训练后如何接入评分 46

7.6 ScorerClient API 速查 47

7.7 配置优先级 48

7.8 查看结果 49

第 8 章　常见问题与排查 50

8.1 环境安装 50

8.2 数据采集 50

8.3 训练 50

8.4 推理与模型控制 50

8.5 评分接入 51

第 9 章　附录：关键文件与命令速查 53

9.1 关键文件 53

9.2 常用命令 54

第 11 章　OrcaGym API 完整参考手册 57

11.1 模块与文件位置 57

11.2 整体关系图 58

11.3 环境生命周期 API（OrcaGymLocalEnv） 58

11.4 模型内省 API（OrcaGymModel） 62

11.5 名称空间解析 API（OrcaGymEnvMixin） 63

11.6 状态查询 API 64

11.7 控制指令 API 66

11.8 相机与视频 API 67

11.9 场景管理 API 69

11.10 控制器适配 API 70

11.11 输入设备 API 72

11.12 日志 API 73

11.13 各模块 API 使用统计 73

11.14 与 OpenPI / 评分系统的关系 74

11.15 使用注意事项 74

第 12 章　相机配置 → 取帧 → OBS 构造 → VLA 推理 完整实践 76

12.1 总体链路 76

12.2 ORCA Studio 相机配置 76

12.3 触发相机推流 77

12.4 连接 WebSocket 取帧 77

12.5 帧预处理与分辨率对齐 78

12.6 构造 OBS 79

12.7 VLA 推理 80

12.8 动作解析与执行 81

12.9 可复用 Python 类模板 82

12.10 常见失败模式与修复 85

12.11 验证相机流是否可用 86

第 13 章　独立机器人 XML 模型与跨平台训练 95

13.1 模型定位：参考实现，不是唯一入口 **95**

13.2 两个 XML 的机器人结构 **95**

13.3 自定义平台训练的工作流 **98**

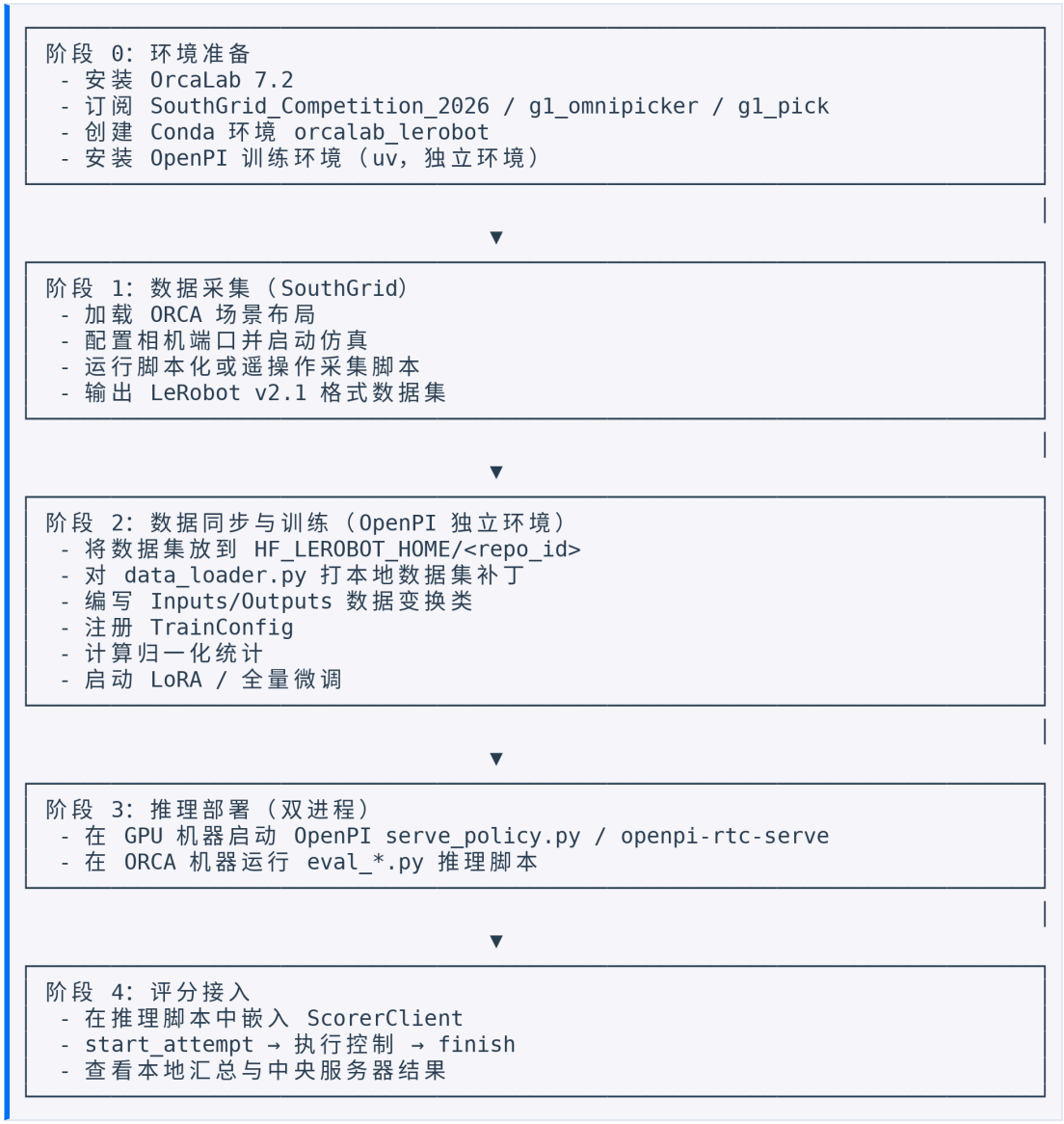
13.4 回到 ORCA 评分：必须满足的条件 **99**

13.5 常见误区 **100**

13.6 推荐路径 **100**

第 1 章

总体流程概览



1.1 代码样例声明

本文档中引用的 `SouthGrid` 仓库代码（采集脚本、推理脚本、控制器、数据存储等）是**大赛官方提供的参考实现/代码样例**，目的是帮助参赛队伍快速理解数据格式、控制接口和评分接入方式。

你可以：

- 直接基于这些脚本进行数据采集、训练、推理和评分；

- 在不改变机器人形态、动作约定、相机端口、数据格式等核心接口的前提下，对代码进行封装、重构或优化；
- 使用自己独立编写的控制算法、规划器或策略网络替代官方脚本，只要最终能向 `ScorerClient` 提交符合评分规则的动作结果。

你不可以：

- **篡改评分规则判定条件**（例如伪造 site 位置、绕过 `ScorerClient` 直接修改 `tasks.yaml`、在评分服务中注入虚假成功帧等）；
- **伪造视频或动作轨迹**（例如使用录屏回放、静止画面、非实时控制等手段欺骗评分服务）；
- **改变机器人形态定义**以获取不公平的判定优势（例如把 `ee_site_right` 改为非末端位置）；
- **绕过 `ScorerClient.start_attempt/finish` 生命周期**，自行构造 attempt 汇总并上传。

评分服务会基于 ORCA 仿真中的真实物理状态、机器人 site 位置以及录屏视频进行综合判定。任何破坏评分细则的行为都可能导致成绩作废。

第 2 章

基础环境搭建

2.1 硬件与系统要求

组件	最低要求	说明
操作系统	Ubuntu 22.04	已验证
GPU（推理）	RTX 4090 / 24 GB 显存	OpenPI 策略服务
GPU（训练 LoRA）	RTX 4090 / 24 GB 显存	$\pi_{0.5}$ LoRA 微调，参考开发者具体使用的模型要求
GPU（视频编码）	RTX 40 系及以上	必须支持 `av1_nvenc`
内存	32 GB 推荐	ORCA + 训练同时运行
磁盘	100 GB 以上	数据集、checkpoint、视频

2.2 安装 OrcaLab 7.2 及以上版本与订阅资产

1. 从 OrcaLab 官方网站下载 **OrcaLab 7.2 及以上版本** 安装包并完成安装。
2. 登录 OrcaLab 资产库，订阅以下三个资产包：
    - `SouthGrid_Competition_2026`
    - `g1_omnipicker`
    - `g1_pick`

> 若场景无法加载或物体不显示，首先检查这三个资产包是否均已订阅。

2.3 克隆 SouthGrid

```
git clone https://github.com/openverse-orca/SouthGrid.git
cd SouthGrid
```

确认根目录存在以下文件：

```
environment-unitree.yml
requirements.txt
scripts/
third_party/
src/
```

2.4 创建 Conda 环境

```
cd /path/to/SouthGrid
conda env create -f environment-unitree.yml
conda activate orcalab_lerobot
bash scripts/install_runtime.sh
```

> 如果你之前用不同的环境名创建过环境（例如 `orcalab`），或者 `environment-unitree.yml` 中的 `name` 字段与上方不同，请使用实际存在的环境名。可用 `conda env list` 查看已创建的环境。

`install_runtime.sh` 会完成以下事情：

- 验证 Python 版本为 3.12.13
- 按哈希锁安装 pip 依赖（`requirements.txt`）
- 安装 `orca-gym==26.7.1` 和 `orca-lab==26.7.1`
- 安装仓库内 `third_party/` 下的 LeRobot 和 OpenPI client
- 执行环境自检（`verify_environment.py`）

禁止单独执行以下命令：

```
pip install pin
pip install pinocchio
pip install casadi
pip install -r requirements.txt
```

2.5 环境验证

```
python scripts/verify_environment.py
```

全部通过后输出：

```
Environment verification OK
```

2.6 安装 OpenPI 训练环境（独立）

在另一台 GPU 服务器或同一台机器的另一个目录中：

```
git clone --recurse-submodules https://github.com/Physical-Intelligence/openpi.git
cd openpi
git checkout 981483d
pip install uv
GIT_LFS_SKIP_SMUDGE=1 uv sync
```

验证 JAX 可见 GPU：

```
uv run python -c "import jax; print(jax.devices())"
```

正常输出类似 `[CudaDevice(id=0)]`。

第 3 章

通用概念与约定

3.1 两套 Python 环境

用途	环境	位置
数据采集、回放、推理客户端	`orcalab_lerobot`（Conda）	`SouthGrid` 目录
模型训练、策略服务	openpi 的 `uv` 环境	`openpi/` 目录

两套环境不共享 Python 包，也不互相依赖。

3.2 机器人形态

本仓库支持两种机器人形态：

形态	场景中的机器人名	移动方式	操作空间
智元 G1 OmniPicker	`g1_omnipicker`	轮式底盘	双臂末端位姿 + 夹爪，18 维
宇树 G1	`g1_pick`	双足行走	28 维关节角

评分系统通过 `robot_id` 参数选择形态。

3.3 LeRobot v2.1 数据集格式

所有采集脚本输出统一格式：

```
<dataset_root>/
├── meta/
│   ├── info.json              # fps, action_dim, 相机键等
│   ├── episodes.jsonl         # 每集 index / length / task
│   ├── episodes_stats.jsonl
│   └── tasks.jsonl            # 语言指令列表
├── data/chunk-000/
│   └── episode_XXXXXX.parquet # action / observation.state / timestamp
└── videos/chunk-000/
    ├── observation.images.cam_head/
    └── observation.images.cam_wrist_r/
```

默认分辨率 480×640，默认帧率 20 FPS，视频编码为 MP4（`av1_nvenc`）。

3.4 动作空间约定

智元 G1 OmniPicker（18 维）：

维度	字段	含义
[0:3]	l_pos_x/y/z	左臂末端位置（base 坐标系，米）
[3:7]	l_quat_x/y/z/w	左臂末端四元数 xyzw
[7:10]	r_pos_x/y/z	右臂末端位置
[10:14]	r_quat_x/y/z/w	右臂末端四元数
[14]	l_gripper_inner_norm	左夹爪内侧归一化
[15]	l_gripper_outer_norm	左夹爪外侧归一化
[16]	r_gripper_inner_norm	右夹爪内侧归一化
[17]	r_gripper_outer_norm	右夹爪外侧归一化

夹爪归一化公式：`norm = (电机值 + 1) / 3`（电机量程 `[-1, 2]`）。

宇树 G1（28 维）：

维度	含义
[0:7]	左臂 7 关节角（rad）
[7:14]	右臂 7 关节角
[14:21]	左手 7 关节角
[21:28]	右手 7 关节角

3.5 相机端口约定

智元 G1 OmniPicker：

相机	OrcaLab 中名称	Color Port

左腕	`camera_wrist_l_color`	7070
右腕	`camera_wrist_r_color`	7080
头部	`camera_head_color`	7090

宇树 G1：

相机	Color Port
头部 `head_cam`	7070
右腕 `wrist_cam`	7080

每路相机必须勾选：**UseNvEnc**、**Color Camera**、**Recording**。

第 4 章

任务二：语言指令按钮操作

4.1 任务目标

任务满分 **40 分**。机器人需要根据语言指令，将末端执行器移动至对应颜色按钮（红/绿/蓝/黄）位置完成模拟按压。接入 `ScorerClient` 后，评分系统会自动判定按钮是否被成功按压，无需开发者关心判定细节。

4.2 选择技术路线

路线	推荐场景	优点	缺点
智元 OmniPicker + 末端位姿策略	有轮式底盘、双臂灵活	动作空间直观，OSC 控制器成熟	需要更多调参
宇树 G1 + 关节角策略	希望直接控制关节	与真实宇树 G1 更接近	需要处理全身平衡
直接关节控制 + HEFT 站立	快速验证评分	已有满分参考实现	不是策略学习路线

4.3 数据采集（智元 OmniPicker）

4.3.1 加载场景

1. 启动 OrcaLab 7.2。
2. 在加载布局对话框中选择： `src/examples/dataCollection/g1_omnipicker/g1_button.json`
3. 按上文配置三路相机端口并勾选 Recording。
4. 点击运行按钮启动仿真。

4.3.2 运行脚本化采集

```
cd /path/to/SouthGrid/src/examples/dataCollection/g1_omnipicker
python g1_omnipicker_collection_scripted_button_lerobot.py \
    --task_config ../common/example.yaml \
    --lerobot_out ~/datasets/g1_button_scripted \
    --repo_id local/g1_omnipicker_button \
    --counts 25,25,25,25 \
    --fps 20 \
    --clock wall
```

参数说明：

参数	含义	默认值	何时修改
`--task_config`	任务配置 YAML	`../common/example.yaml`	使用其他配置时
`--lerobot_out`	数据集输出目录	无，必须指定	每次采集都要改
`--repo_id`	数据集仓库名	`local/g1_omnipicker_button`	区分不同数据集
`--counts`	红/绿/黄/蓝各采集集数	交互式询问	批量采集时指定
`--fps`	帧率	20	需要改帧率时
`--clock`	时钟源：sim/wall	`wall`	与遥操作数据对齐时
`--resume`	断点续采	未启用	续采时追加

采集过程：

- 从 `pose_g1_button_candidates.yaml` 读取每个按钮的候选接触位姿。
- 每集生成 4 段轨迹：接近 → 前推 → 保压 → 后撤。
- 左臂锁定，右臂用 OSC 控制器跟踪目标位姿。
- 数据写入 LeRobot v2.1。

4.3.3 遥操作采集（可选）

如果需要更高质量的人类示范数据：

```
adb reverse tcp:8001 tcp:8001

python g1_omnipicker_collection_tele_lerobot.py \
    --task_config ../common/example.yaml \
    --lerobot_out ~/datasets/g1_tele_button \
    --repo_id local/g1_omnipicker_button_tele \
    --task "按红色按钮" \
    --fps 20 \
    --clock wall \
    --cameras head,wrist_r \
    --camera_source websocket
```

Pico 手柄按键映射：

- 右臂末端位姿：右手柄 6DOF
- 左夹爪：左 X/Y 或左扳机
- 右夹爪：右 A/B 或右扳机
- 开始当前集：第一次按左侧握键
- 结束并保存：第二次按左侧握键
- 放弃本集：按右侧握键
- 终止全部：左右握键同时按下

4.4 数据采集（宇树 G1）

> 宇树 G1 当前交付范围仅包含数据采集功能，暂不提供策略训练和在线推理集成。

4.4.1 加载场景

1. 启动 OrcaLab 7.2。
2. 加载布局： `src/examples/dataCollection/unitree_g1/g1_pick_buttons.json`
3. 确认场景中的机器人名称为 `g1_pick`。
4. 配置头部 7070、右腕 7080 两路相机。

4.4.2 遥操作采集

```
conda activate orcalab_lerobot
cd /path/to/SouthGrid/src/examples/dataCollection/unitree_g1

adb reverse tcp:8001 tcp:8001

python -u g1_pick_osc_collection_tele_lerobot.py \
    --level default \
    --task_config ../common/example.yaml \
    --scene_json g1_pick_buttons.json \
    --task "按红色按钮" \
    --lerobot_out ~/datasets/g1_pick_tele_button \
    --repo_id local/g1_pick_button_tele \
    --fps 20 --clock wall \
    --cameras head,wrist_r \
    --orcagym_addr localhost:50051 \
    --agent_name g1_pick
```

Pico 手柄按键映射：

- 右臂末端位姿：右手柄 6DOF
- 左夹爪：左 X/Y 或左扳机
- 右夹爪：右 A/B 或右扳机
- 开始当前集：第一次按左侧握键
- 结束并保存：第二次按左侧握键
- 放弃本集：按右侧握键
- 终止全部：左右握键同时按下

4.5 数据检查

采集完成后，检查数据集结构：

```
ls ~/datasets/g1_button_scripted/
# 应包含 meta/ data/ videos/

python - <<'PY'
from pathlib import Path
from lerobot.common.datasets.lerobot_dataset import LeRobotDataset
import os
os.environ["HF_LEROBOT_HOME"] = str(Path.home() / "datasets")
ds = LeRobotDataset("local/g1_omnipicker_button")
print("episodes:", ds.num_episodes)
print("frames:", ds.num_frames)
print("tasks:", ds.meta.tasks)
PY
```

4.6 模型训练（OpenPI）

4.6.1 同步数据集到训练机器

将采集好的数据集复制到训练机器的：

```
HF_LEROBOT_HOME/local/g1_omnipicker_button/
```

设置环境变量（可加入 `~/.bashrc`）：

```
export HF_LEROBOT_HOME=/data/lerobot_datasets
export OPENPI_DATA_HOME=/data/openpi_cache
```

4.6.2 打本地数据集补丁

openpi 默认从 HuggingFace Hub 加载数据集，需要修改 `src/openpi/training/data_loader.py`，使其优先读取本地 `HF_LEROBOT_HOME`。

补丁内容：

```
from lerobot.common.constants import HF_LEROBOT_HOME
_local_path = HF_LEROBOT_HOME / repo_id
_root = str(_local_path) if _local_path.exists() else None

if _root is not None:
    import json as _json
    from pathlib import Path as _Path
    _info_path = _Path(_root) / "meta" / "info.json"
    _local_fps = _json.loads(_info_path.read_text()).get("fps", 30) if
_info_path.exists() else 30
    dataset = lerobot_dataset.LeRobotDataset(
        data_config.repo_id,
        root=_root,
        delta_timestamps={
            key: [t / _local_fps for t in range(action_horizon)] for key in
data_config.action_sequence_keys
        },
    )
    dataset_meta = None
else:
    dataset_meta = lerobot_dataset.LeRobotDatasetMetadata(repo_id)
    dataset = lerobot_dataset.LeRobotDataset(
        data_config.repo_id,
        delta_timestamps={
            key: [t / dataset_meta.fps for t in range(action_horizon)] for key in
data_config.action_sequence_keys
        },
    )
```

4.6.3 编写数据变换类

在 openpi 中创建策略文件，例如 `src/openpi/policies/g1_omnipicker_policy.py`：

```
import dataclasses
import numpy as np
from openpi.policies import transforms

@dataclasses.dataclass(frozen=True)
class G1OmnipickerInputs(transforms.DataTransformFn):
    action_dim: int = 18
    action_horizon: int = 50

    def __call__(self, data: dict) -> dict:
        return {
            "state": data["observation.state"],
            "images": {
                "cam_head": data["observation.images.cam_head"],
                "cam_wrist_r": data["observation.images.cam_wrist_r"],
            },
            "prompt": data.get("prompt", ""),
            "actions": data["action"][:self.action_horizon],
        }

@dataclasses.dataclass(frozen=True)
class G1OmnipickerOutputs(transforms.DataTransformFn):
    def __call__(self, data: dict) -> dict:
        return {"actions": data["actions"]}
```

4.6.4 定义数据集配置

在 `src/openpi/training/config.py` 中：

```
@dataclasses.dataclass(frozen=True)
class G1OmnipickerButtonDataConfig(DataConfigFactory):
    repo_id: str = "local/g1_omnipicker_button"

    def create(self, model_config) -> DataConfig:
        return DataConfig(
            repo_id=self.repo_id,
            input_transform=G1OmnipickerInputs(
                action_dim=model_config.action_dim,
                action_horizon=model_config.action_horizon,
            ),
            output_transform=G1OmnipickerOutputs(),
            prompt_from_task=True,
        )
```

4.6.5 注册 TrainConfig

```
TrainConfig(
    name="pi05_g1_omnipicker_button_lora",
    model=pi0_config.Pi0Config(
        pi05=True,
        action_dim=18,
        action_horizon=50,
        paligemma_variant="gemma_2b_lora",
        action_expert_variant="gemma_300m_lora",
    ),
    data=G1OmnipickerButtonDataConfig(),
    batch_size=32,
    num_train_steps=10_000,
    lr_schedule=_optimizer.CosineDecaySchedule(
        warmup_steps=1_000,
        peak_lr=2e-4,
        decay_steps=10_000,
        decay_lr=2e-5,
    ),
    optimizer=_optimizer.AdamW(clip_gradient_norm=1.0),
    freeze_filter=pi0_config.Pi0Config(
        pi05=True,
        action_dim=18,
        action_horizon=50,
        paligemma_variant="gemma_2b_lora",
        action_expert_variant="gemma_300m_lora",
    ).get_freeze_filter(),
    ema_decay=None,
    weight_loader=weight_loaders.CheckpointWeightLoader(
        ".cache/openpi/openpi-assets/checkpoints/pi05_base/params"
    ),
    num_train_steps=10_000,
    log_interval=100,
    save_interval=5_000,
)
```

4.6.6 计算归一化统计

```
cd /path/to/openpi
HF_LEROBOT_HOME=/data/lerobot_datasets \
uv run scripts/compute_norm_stats.py --config-name pi05_g1_omnipicker_button_lora
```

4.6.7 启动训练

```
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
HF_LEROBOT_HOME=/data/lerobot_datasets \
XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 \
WANDB_MODE=offline \
uv run scripts/train.py pi05_g1_omnipicker_button_lora \
    --exp-name=exp01 \
    --overwrite
```

checkpoint 保存路径：

```
checkpoints/pi05_g1_omnipicker_button_lora/exp01/<step>/
```

4.7 推理部署

4.7.1 启动策略服务器

在 GPU 机器上：

```
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py \
    --port 8010 \
    policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_button_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_button_lora/exp01/9999
```

等待日志输出：

```
server listening on 0.0.0.0:8010
```

4.7.2 运行推理（智元）

```
conda activate orcalab_lerobot
cd /path/to/SouthGrid/src/examples/inference/g1_omnipicker

python eval_g1_omnipicker_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost \
    --port 8010 \
    --prompt "按红色按钮" \
    --max_steps 500 \
    --action_repeat 1 \
    --episodes 3
```

4.7.3 运行推理（宇树）

> 宇树 G1 的在线推理当前不在 SouthGrid 交付范围内，暂不提供策略训练和在线推理集成。

第 5 章

任务三：电力工具归纳收置

5.1 任务目标

任务满分 **40 分**。机器人需要依次将 5 件电力工具（验电笔、螺丝刀、扳手、小手电筒、电工刀）从桌面拾取并归位到工具箱中。接入 `ScorerClient` 后，评分系统会自动判定每件工具是否归位成功，无需开发者关心判定细节。

场景中工具对应的 site 名（供参考）：

工具	ORCA site 名
验电笔	`Group_Interactive_TestPencil_task_testpencil_testpoint_site`
螺丝刀	`Group_Interactive_Screwdriver_task_screwdriver_site`
扳手	`Group_Interactive_Spanner_task_spanner_site`
小手电筒	`Group_Interactive_Flashlight_task_flashlight_site`
电工刀	`Group_Interactive_ElectriciansKnife01_task_electriciansknife01_site`

5.2 数据采集（智元 OmniPicker）

5.2.1 加载场景

1. 启动 OrcaLab 7.2。
2. 加载布局： `src/examples/dataCollection/g1_omnipicker/g1_tool.json`
3. 配置三路相机（与按钮任务相同）。

5.2.2 运行脚本化采集

```
cd /path/to/SouthGrid/src/examples/dataCollection/g1_omnipicker
python g1_omnipicker_collection_scripted_tool_lerobot.py \
    --task_config ../common/example.yaml \
    --lerobot_out ~/datasets/g1_tool_scripted \
    --repo_id local/g1_omnipicker_tool \
    --num_episodes 20 \
    --fps 20
```

关键参数：

参数	说明	默认值
`--num_episodes`	采集轮数	1
`--num_tools`	每集抓取工具数量	5
`--tools`	指定抓取工具编号	空
`--randomize`	随机排列槽位	未启用
`--speed`	轨迹整体提速倍率	1.0
`--waypoint_files`	5 个路点 YAML 路径	同目录 `my_waypoint_tool1.yaml` 至 `my_waypoint_tool5.yaml`

轨迹节奏参数：

```
--safe_z 0.50
--steps_transit 50
--steps_descend 30
--steps_grasp 35
--steps_settle 30
--steps_lift 35
--steps_place_via 45
--steps_to_box 50
--steps_release 45
--steps_release_settle 20
--steps_lift_after 15
```

5.3 模型训练（OpenPI）

流程与任务二一致，区别：

- 数据集：`local/g1_omnipicker_tool`
- prompt：`"整理工具"`
- action_dim：18（智元）
- action_horizon：根据任务节奏调整，通常保持 50

示例配置：

```
TrainConfig(
    name="pi05_g1_omnipicker_tool_lora",
    model=pi0_config.Pi0Config(
        pi05=True,
        action_dim=18,
        action_horizon=50,
        paligemma_variant="gemma_2b_lora",
        action_expert_variant="gemma_300m_lora",
    ),
    data=G1OmnipickerToolDataConfig(repo_id="local/g1_omnipicker_tool"),
    batch_size=32,
    num_train_steps=30_000,
    ...
)
```

5.4 推理部署

5.4.1 启动策略服务器

```
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py \
    --port 8010 \
    policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_tool_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_tool_lora/exp01/29999
```

5.4.2 运行推理

```
conda activate orcalab_lerobot
cd /path/to/SouthGrid/src/examples/inference/g1_omnipicker

python eval_g1_omnipicker_tool_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost --port 8010 \
    --prompt "整理工具" \
    --max_steps 10000 \
    --episodes 1
```

该脚本特点：

- 默认锁定左臂，减少策略自由度；
- 支持 `--grasp_integral` 近桌外环积分；
- `max_steps` 默认 10000，适合长程任务。

5.5 任务三评分脚本（需自行实现）

目前仓库中没有现成的 `run_task3_tool_storage.py`。参考 Task 2 的实现，最小评分接入代码如下：

```
from orca_scorer_client import ScorerClient

scorer = ScorerClient(
    team_id="team_xxx",
    team_token="token_team_xxx",
    robot_id="g1_omnipicker",
)

info = scorer.start_attempt("task3_tool_storage")
print(f"attempt started: {info.get('attempt_id')}")

# 在这里运行你的策略或脚本化控制
# 依次把 5 件工具放到工具箱基座 0.3m 内并保持 1 秒以上

summary = scorer.finish()
print(f"score: {summary.get('score')} / {summary.get('max_score')}")
for r in summary.get("step_results", []):
    status = "PASS" if r.get("success") else "FAIL"
    print(f"  {r.get('step_id')}: {status}")
```

建议将 `eval_g1_omnipicker_tool_lerobot.py` 改造：

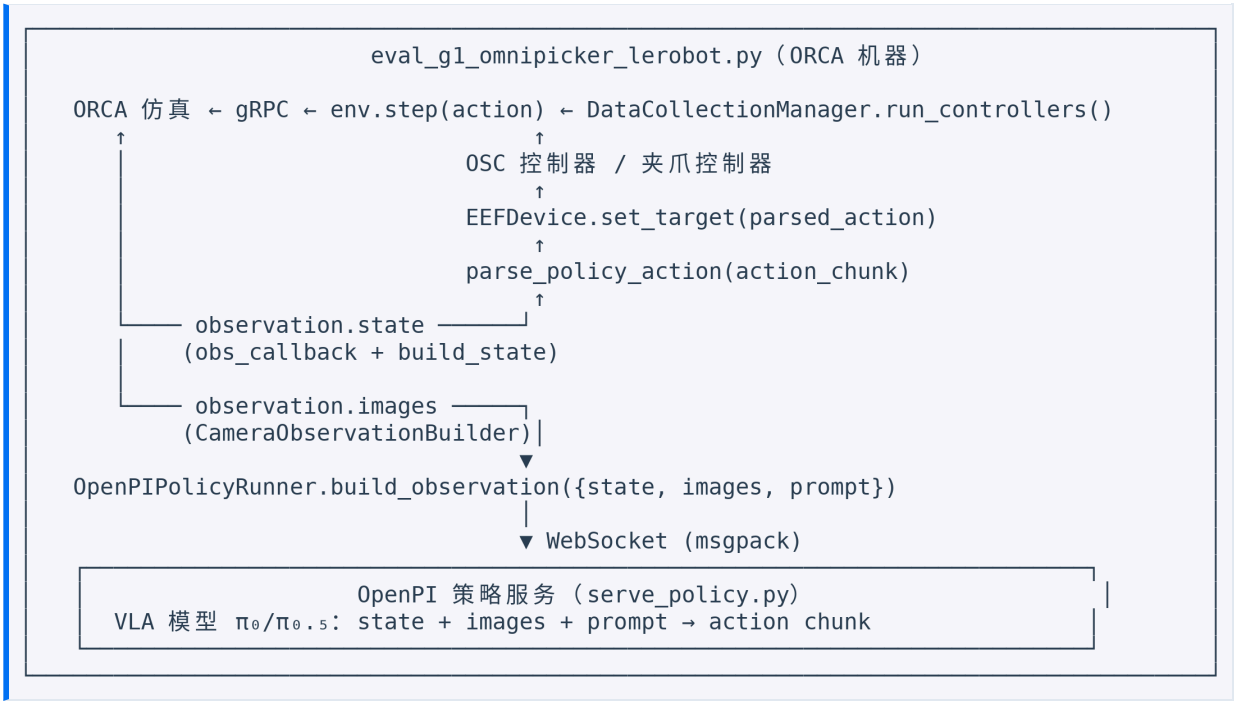
1. 在策略循环前调用 `start_attempt`；
2. 在策略循环结束后调用 `finish`；
3. 可选：每完成一件工具后查询 `scorer.status()` 确认步骤通过。

第 6 章

训练后的模型如何控制 ORCA 机器人

训练完成后，checkpoint 并不能直接驱动 ORCA 仿真中的机器人。完整控制链路由**策略服务（GPU 侧）**和**推理客户端（ORCA 侧）**两个进程配合完成。下面以智元 G1 OmniPicker 为例，详细说明从 checkpoint 到机器人关节指令的每一步。

6.1 整体链路



6.2 策略服务进程（OpenPI 侧）

```
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py \
    --port 8010 \
    policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_button_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_button_lora/exp01/9999
```

服务启动后：

- 加载 checkpoint 中的模型权重；
- 监听 `0.0.0.0:8010`；
- 对每个推理请求，返回 `actions`（形状 `[action_horizon, action_dim]`）。

6.3 推理客户端进程（ORCA 侧）

`eval_g1_omnipicker_lerobot.py` 的核心职责：

6.3.1 创建 ORCA 仿真环境

```
manager = DataCollectionManager(
    agent_name="g1_omnipicker",
    env_name="DataCollection",
    entry_point="envs.dataCollection.dataCollection_env:DataCollectionEnv",
    default_joint_values=build_default_joint_values(),
    obs_callback=storage.obs_callback,
    env_index=0,
    device=None,
    scene_manager=scene_manager,
    frame_skip=5,
    orcagym_addr="localhost:50051",
)
env = manager.env
manager.set_disable_actuator_group([agent_conf.positions_group])
manager.mode = DataCollectionManager.DataCollectionMode.INFERENCE
```

- `orcagym_addr="localhost:50051"` 指向本地 ORCA gRPC 服务；
- `frame_skip=5` 表示客户端每调用一次 `env.step()`，ORCA 内部推进 5 个物理步；
- `time_step=0.001`，因此 `real_time_step = 0.001 * 5 = 0.005s`，控制循环最高可跑到 200Hz；
- 实际策略执行频率由 `action_repeat` 决定：`action_repeat=10` 时每个模型动作持续 10 个 env step（0.05s），对应 20Hz，与采集数据频率一致。

6.3.2 挂载控制器

```
l_arm  = create_arm(env, agent_conf.l_arm)
r_arm  = create_arm(env, agent_conf.r_arm)
l_grip = create_gripper(env, agent_conf.gripper_l)
r_grip = create_gripper(env, agent_conf.gripper_r)
manager.add_controller(l_arm)
manager.add_controller(r_arm)
manager.add_controller(l_grip)
manager.add_controller(r_grip)
```

- `l_arm` / `r_arm` 是 **OSC（Operational Space Control）** 臂控器；
- 输入：目标末端位置 + 四元数（base 坐标系）；
- 输出：关节位置/力矩控制指令，写入 `env.ctrl`；
- `l_grip` / `r_grip` 是 `Controller2F85Reverse` 夹爪控制器，输入 `[inner, outer]` 电机绝对值。

6.3.3 连接策略服务器

```
policy_runner = OpenPIPolicyRunner(
    host=args.host,
    port=args.port,
    prompt=args.prompt,
    camera_name_map=camera_name_map,
    cameras=_shared_cameras,
    target_hw=_target_hw,
    use_images=not args.no_images,
)
```

内部使用 `openpi_client.websocket_client_policy.WebsocketClientPolicy` 建立 WebSocket 连接。

6.3.4 相机视频观测的获取（关键实践细节）

推理时的图像不是通过 `obs_callback` 读取的，而是由 `OpenPIPolicyRunner` 内部的 `CameraObservationBuilder` 独立从 WebSocket 内存流拉取。

完整启动流程（必须在 ORCA 场景加载并渲染后才能执行）：

```
# 1. 通知 ORCA Studio 开始推流
camera_map = omnipicker_camera_map(enable_wrist_l=args.enable_wrist_l)
os.makedirs(STREAM_TRIGGER_PATH, exist_ok=True)
env.begin_save_video(STREAM_TRIGGER_PATH)   # 触发相机 WebSocket 推流

# 2. 等待端口就绪并连接 CameraWrapper
_shared_cameras = bring_up_cameras(camera_map, port_timeout=30.0,
frame_timeout=30.0)
_target_hw = probe_camera_hw(_shared_cameras, camera_map)

# 3. 创建策略 runner，runner 内部持有 CameraObservationBuilder
policy_runner = OpenPIPolicyRunner(
    host=args.host,
    port=args.port,
    prompt=args.prompt,
    camera_name_map=camera_name_map,   # env 传感器名 → 策略键名
    cameras=_shared_cameras,
    target_hw=_target_hw,
    use_images=not args.no_images,
)
```

每步取帧与预处理：

```
# OpenPIPolicyRunner.build_observation
images = self.cam_builder.build_images() if self.use_images else {}
return {"state": state, "images": images, "prompt": self.prompt}
```

`CameraObservationBuilder.build_images()` 做的事：

1. 对每个配置的相机，调用 `CameraWrapper.get_frame(format="rgb24")` 取最新帧；
2. 若分辨率与训练不一致，用 `cv2.INTER_AREA` 缩放到 `(480, 640)`；
3. 把 `(H, W, 3)` 的 RGB 图像转置为 `(3, H, W)`；
4. 若某路相机掉线，用全黑图占位，避免策略崩溃。

相机配置要求（ORCA Studio 中必须完成）：

相机	传感器名	WebSocket 端口	说明
头部	`camera_head_color`	7090	默认启用
右腕	`camera_wrist_r_color`	7080	默认启用
左腕	`camera_wrist_l_color`	7070	需 `--enable_wrist_l`

每个相机组件必须勾选：

- `UseNvEnc`
- `Color Camera`
- `Recording`

并且在场景渲染前，三路相机都要完成配置再统一开始录制。

常见失败模式：

现象	原因	处理
`bring_up_cameras` 超时	ORCA 没开始渲染 / 端口未监听 / 相机未勾选 Recording	检查 ORCA 窗口是否在渲染，检查端口 `netstat -anp | grep 7080`
首帧一直等不到	WebSocket 连接在端口就绪前建立，后台线程已退出	`bring_up_cameras` 已内置 TCP 探测 + 线程重启，通常可自愈
图像是全黑的	`begin_save_video` 未调用或调用失败	确认 `/tmp/eval_g1_lerobot_stream` 目录可写，且 ORCA 推流已触发
策略输出异常	训练/推理图像 key 不一致	确保 `camera_name_map` 与训练数据集 `features` 中的相机键一致

6.3.5 构建观测

每控制周期：

```
state = storage.build_state(storage.obs_callback(env))
action_chunk = policy_runner.infer_action_chunk(state)
```

`storage.obs_callback(env)` 从 ORCA 环境中读取**本体状态**：

- 双臂末端位姿（由正运动学计算，base 坐标系）；
- 夹爪电机编码器值；
- 臂部关节位置（用于控制器内部）。

注意：相机图像不由 `obs_callback` 读取，而是由 `OpenPIPolicyRunner.build_observation()` 在每次推理时独立拉取。

`storage.build_state` 把本体观测拼接成 18 维 `observation.state`：

```
[0:3]   左臂位置
[3:7]   左臂四元数 xyzw
[7:10]  右臂位置
[10:14] 右臂四元数 xyzw
[14:16] 左夹爪归一化 [inner, outer]
[16:18] 右夹爪归一化 [inner, outer]
```

6.3.6 VLA 推理请求与响应

`policy_runner.infer_action_chunk(state)` 内部完成的 VLA 推理流程：

```
observation = {
    "state": state,                        # np.ndarray, shape=(18,), dtype=float32
    "images": {                            # 由 CameraObservationBuilder 提供
        "cam_head": np.ndarray(shape=(3, 480, 640), dtype=uint8),
        "cam_wrist_r": np.ndarray(shape=(3, 480, 640), dtype=uint8),
    },
    "prompt": "按红色按钮",                # 与训练时一致的语言指令
}

# 1. msgpack + numpy 编码
packed = msgpack_numpy.Packer().pack(observation)

# 2. WebSocket 发送到 OpenPI 策略服务
self._ws.send(packed)

# 3. 等待模型推理结果
response = self._ws.recv()
result = msgpack_numpy.unpackb(response)

# 4. 解析 action chunk
actions = np.asarray(result["actions"], dtype=np.float32)
# actions shape: (action_horizon, action_dim)
```

实际运行中的关键点：

- 每次调用 `infer_action_chunk` 都会发送一次 WebSocket 请求；
- OpenPI 服务端返回的是未来 `action_horizon` 步的动作序列；
- 客户端可以只执行 chunk 中的第 0 步，也可以像本脚本一样顺序执行整个 chunk 再重新推理；
- 如果 `--no_images` 开启，`images` 会被置为空 dict，此时策略退化为纯 state + prompt 输入（必须与训练时一致）。

6.3.7 解析并执行动作

```
for model_action in action_chunk:
    parsed_action = parse_policy_action(model_action)
    device.set_target(**action_dict_for_apply(parsed_action))

    for _ in range(args.action_repeat):
        action = manager.run_controllers()
        env.step(action)
        env.render()
```

动作解析：

策略输出	字段	后续处理
`[0:3]`	`l_pos_b`	左臂 OSC 目标位置
`[3:7]`	`l_quat_b`	左臂 OSC 目标姿态（xyzw 四元数）
`[7:10]`	`r_pos_b`	右臂 OSC 目标位置
`[10:14]`	`r_quat_b`	右臂 OSC 目标姿态
`[14:16]`	左夹爪归一化	反归一化为电机绝对值，`Controller2F85Reverse.update_ctrl`
`[16:18]`	右夹爪归一化	反归一化为电机绝对值

`action_dict_for_apply` 中夹爪反归一化：

```
val = norm * (hi - lo) + lo
# 默认量程 (-1, 2)，即 val = norm * 3 - 1
```

6.3.8 控制频率与实时性

- 默认 `action_repeat=1`，策略动作每个 env step 更新一次；增大 `action_repeat` 可降低策略执行频率、给 OSC 更多收敛时间；
- 若 `--sleep` 开启，每次 `env.step` 后会按 `real_time_step=0.005s` 睡眠，保证 wall-clock 与仿真时间同步；
- 对 Task 2 按钮按压，建议在 hold 阶段使用 30Hz 以上 render 频率，让评分服务采样到足够帧。

6.4 宇树 G1 的控制差异

> 宇树 G1 的在线推理当前不在 SouthGrid 交付范围内。以下内容供需要自行实现宇树 G1 推理的开发者参考。

宇树 G1 的推理脚本不经过 OSC，而是使用**关节位置控制器 + Δq 开环积分**。

动作空间约定：

- 采集时 `build_action(state_prev, state_cur)` 返回 `Δq = q[t+1] - q[t]`；
- 因此策略学习的是**关节角增量**，不是绝对关节角；
- 推理时把 Δq 累加到当前目标 `q_cmd` 上：

```
state = storage.build_state(storage.obs_callback(env))   # 当前 28 维关节角 q
action_chunk = policy_runner.infer_action_chunk(state)   # 输出 Δq chunk

for model_action in action_chunk:
    dq = np.asarray(model_action, dtype=np.float32).reshape(-1)[:STATE_DIM]
    q_cmd = q_cmd + dq                                    # 开环积分
    device.set_target(q_cmd[:ARM_DIM], q_cmd[ARM_DIM:STATE_DIM])

    for _ in range(args.action_repeat):
        action = manager.run_controllers()
        env.step(action)
        env.render()
```

关键注意点：

1. `q_cmd` 在每集开始时用当前观测初始化，之后纯累加策略输出的 Δq；
2. `ReplayPositionController` 把 `q_cmd` 作为目标关节角，通过位置执行器跟踪；
3. 这种开环方式对策略漂移敏感：如果 Δq 有系统偏差，`q_cmd` 会逐步偏离真实状态；
4. 可通过 `--action_repeat` 让电机有更多时间跟踪目标，或通过 `--grasp_integral` 等参数启用低增益积分补偿。

与智元 OmniPicker 的核心区别：

项目	智元 OmniPicker	宇树 G1
动作含义	绝对末端位姿 + 夹爪	关节角增量 Δq
控制器	OSC	`ReplayPositionController`

执行方式	直接设为 OSC 目标	`q_cmd += Δq` 开环积分
漂移风险	低（每次直接给出绝对目标）	高（长期积分可能累积误差）

6.5 关键调试信息

推理脚本会周期性输出：

```
[PROF] n=50  ctrl=1.2ms  env.step=2.1ms  render=0.8ms  preview=0.5ms | total≈4.6ms
step=0100/0500  cmd_L=[+0.312,-0.102,+0.587]  cmd_R=[+0.521,+0.113,+0.498]  grip_L=
[-0.500,1.200]  grip_R=[-0.856,1.500]
```

- `cmd_L/cmd_R` 是下发给 OSC 控制器的目标末端位置（智元 OmniPicker）；
- `grip_L/grip_R` 是反归一化后的夹爪电机绝对值；
- 宇树 G1 的调试输出会打印 `q_cmd` 与 `dq` 的 L2 范数，用于监控开环积分漂移；
- 若 `total` 接近或超过 `real_time_step`，说明控制循环有掉帧风险，需关闭预览或降低图像分辨率。

第 7 章

评分系统接入

7.1 评分客户端安装

评分客户端 `orca_scorer_client` 是 `competition_scoring_client` 包的一部分。该包不发布到 PyPI，需要通过华为云私有 Git 仓库直接安装，**禁止通过本地源码目录 `pip install -e .` 安装**。

```
conda activate orcalab_lerobot

pip install git+https://codehub.devcloud.cn-east-
3.huaweicloud.com/bbb453d36204472ca668f7c8ca786e59/competition_scoring_client.git@So
uthGrid
```

> 注意：不要通过克隆仓库到本地并 `pip install -e .` 的方式安装，应始终使用上方的 `pip install git+...` 命令。

安装完成后验证：

```
python -c "from orca_scorer_client import ScorerClient;
print(ScorerClient.__module__)"
```

应输出包路径，无 ImportError。

视频录制依赖安装（按操作系统）

`competition_scoring_client` 安装后，评分核心功能已可用；但**默认启用的视频录制**还依赖以下组件。不同操作系统需要安装的依赖不同，请根据你的系统选择对应章节。

> 如果你只想本地调试，可设置环境变量禁用视频：`export ORCA_SCORING_VIDEO_ENABLED=0`。禁用后评分判定仍可正常进行，但中央服务器不会收到任务视频。

通用依赖（所有系统）

以下 Python 包需要在 `orcalab_lerobot` 环境中安装：

```
pip install imageio-ffmpeg mss
```

包	作用	是否必须
`imageio-ffmpeg`	调用 ffmpeg 进行视频转码（H.264 + yuv420p）	**必须**
`mss`	屏幕捕获	建议

以下系统工具所有系统都需要：

```
sudo apt-get update
sudo apt-get install -y ffmpeg
```

工具	作用	是否必须
`ffmpeg`	视频编码、转码、封装 MP4	**必须**

Ubuntu 22.04（X11 窗口系统）

Ubuntu 22.04 默认使用 X11，需要额外安装以下组件：

Python 包：

```
pip install pyautogui python-xlib
```

包	作用	是否必须
`pyautogui`	X11 屏幕捕获（首选，权限处理更好）	**强烈建议**
`python-xlib`	X11 窗口查找、置顶、恢复	**必须**

系统工具：

```
sudo apt-get install -y xdotool wmctrl
```

工具	作用	是否必须
`xdotool`	窗口查找、置顶、调整大小、恢复位置	**强烈建议**
`wmctrl`	窗口管理后备	建议

> 实践经验：Linux 下 `mss` 常因 X11 权限问题导致捕获失败，因此评分服务会优先使用 `pyautogui`。务

> 必将 `pyautogui` 装好。

快速自检：

```
python -c "import pyautogui, Xlib, imageio_ffmpeg; print('OK')"
which ffmpeg && which xdotool
```

Ubuntu 24.04（Wayland 窗口系统）

Ubuntu 24.04 默认使用 Wayland，X11 工具（`pyautogui`、`python-xlib`、`xdotool`）无法直接使用，需要 Wayland 原生工具：

Python 包：

```
pip install pyautogui python-xlib
```

> 虽然 Wayland 下 `pyautogui` 和 `python-xlib` 的屏幕捕获功能受限，但评分服务内部会检测窗口系统并回退到 `mss` 或其他可用方案，因此这些包仍需安装以提供基础支持。

系统工具：

```
sudo apt-get install -y grim slurp
```

工具	作用	是否必须
`grim`	Wayland 截图工具	**必须**
`slurp`	Wayland 区域选择工具	**必须**

快速自检：

```
python -c "import pyautogui, Xlib, imageio_ffmpeg, mss; print('OK')"
which ffmpeg && which grim && which slurp
echo $XDG_SESSION_TYPE  # 应输出 wayland
```

> **注意**：如果 Ubuntu 24.04 上视频录制仍不稳定，可考虑切换到 X11 会话（登录界面点击齿轮图标选择 "Ubuntu on Xorg"），然后按 Ubuntu 22.04 的依赖安装即可。这是目前最稳定的视频录制方案。

一键安装（推荐）

如果你不确定自己的系统配置，直接运行以下命令安装全部依赖：

```
# Python 包
pip install pyautogui python-xlib imageio-ffmpeg mss

# 系统工具（自动检测并安装对应窗口系统的工具）
sudo apt-get update
sudo apt-get install -y ffmpeg xdotool wmctrl grim slurp
```

最终自检：

```
python -c "import pyautogui, Xlib, imageio_ffmpeg, mss; print('Python deps OK')"
which ffmpeg && echo "ffmpeg OK"
```

如果缺少其中任何一项，视频录制可能失败，中央服务器将无法看到任务视频。

7.2 ScorerClient 与 ORCA 的部署关系

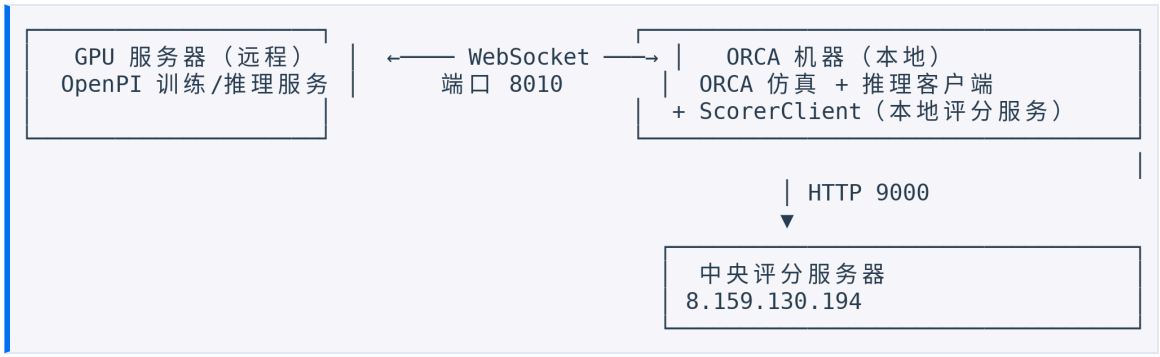
评分客户端（`ScorerClient` 及其自动启动的本地评分服务）必须与 ORCA 仿真运行在同一台物理机上。 原因如下：

1. **gRPC 连接本地 ORCA**：`ScorerClient` 默认 `orca_addr=localhost:50051`，评分服务需要通过 gRPC 读取 ORCA 中机器人 site、body 的真实世界坐标；
2. **屏幕视频录制**：评分服务默认通过 `pyautogui` 捕获本机屏幕，录制 ORCA 窗口作为任务视频上传中央服务器。若 ORCA 在远程机器上，本地录屏将看不到任务画面，导致视频与状态不匹配；
3. **自动启动**：`ScorerClient` 会自动管理本地评分服务的生命周期（启动/关闭）。

> 理论上，如果你手动禁用视频、手动启动评分服务并将其 `orca_addr` 指向远程 ORCA，可以把评分服务与 ORCA 分开部署，但这**不是官方支持的标准流程**，大赛提交时也不推荐这样做。

策略服务器（OpenPI `serve_policy.py`）可以部署在远程 GPU 服务器上。 推理客户端通过 WebSocket 连接策略服务器，两者只需网络可达即可。

因此典型部署拓扑为：



多评分服务端口与禁用视频

- 默认本地评分服务端口为 **9000**。如果该端口被残留进程占用，可换用 **9001–9005** 等独立端口，并在 `ScorerClient` 中通过 `base_url` 指定：

```
scorer = ScorerClient(
    team_id="team_xxx",
    team_token="token_team_xxx",
    base_url="http://127.0.0.1:9001",
)
```

- 如果你只想本地调试，可设置环境变量禁用视频：

```
export ORCA_SCORING_VIDEO_ENABLED=0
```

  禁用视频后，评分判定仍可正常进行，但中央服务器不会收到任务视频。

7.3 本地推理评分接入

当策略服务器与 ORCA 在同一台机器时，推理与评分接入最简单。下面命令假设你已经按 7.5 模型训练后如何接入评分 的示例，在 `eval_g1_omnipicker_lerobot.py` 中添加了 `--team-id`、`--team-token` 等参数并嵌入了 `ScorerClient`：

```
# 终端 1：启动策略服务（本地 GPU）
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py --port 8010 policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_button_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_button_lora/exp01/9999

# 终端 2：运行带评分接入的推理脚本
conda activate orcalab_lerobot
cd /path/to/SouthGrid/src/examples/inference/g1_omnipicker
python eval_g1_omnipicker_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost --port 8010 \
    --prompt "按红色按钮" \
    --team-id team_xxx --team-token token_team_xxx \
    --robot-id g1_omnipicker \
    --task-id task2_button_press \
    --targets red green blue yellow
```

> 原始 `eval_g1_omnipicker_lerobot.py` 没有 `--team-id` 等参数，直接运行上述命令会报错。必须先按 7.5 节进行改造。

在推理脚本中，评分生命周期如下：

```
from orca_scorer_client import ScorerClient

# 1. 初始化：自动启动本地评分服务（若未运行）
scorer = ScorerClient(
    team_id=args.team_id,
    team_token=args.team_token,
    robot_id=args.robot_id,
    timeout=60.0,
)

# 2. 开始尝试：评分服务开始录制屏幕、采样 ORCA 状态
info = scorer.start_attempt(
    args.task_id,
    targets=args.targets,
    prompt=args.prompt,
)
print(f"[scorer] attempt started: {info.get('attempt_id')}")

try:
    # 3. 执行你的推理/控制循环
    #    评分服务会在后台自动判定每一步是否成功
    run_policy_inference_loop(...)
finally:
    # 4. 结束尝试：停止录制、计算总分、上传视频
    summary = scorer.finish()
    print(f"[scorer] score: {summary.get('score')} / {summary.get('max_score')}")

    # 5. 等待视频上传完成（默认启用视频时）
    time.sleep(90)
```

7.4 远程推理评分接入

如果 OpenPI 训练/推理服务在远程 GPU 服务器，而 ORCA 在本地机器：

步骤 1：在远程 GPU 服务器启动策略服务

```
# 在 gpu_server 上执行
ssh user@gpu_server
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py \
    --host 0.0.0.0 \
    --port 8010 \
    policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_button_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_button_lora/exp01/9999
```

注意：OpenPI 默认 `--host` 可能是 `localhost`，远程访问时需显式 `--host 0.0.0.0`。

步骤 2：在本地 ORCA 机器建立 SSH 隧道

```
ssh -N -L 8010:localhost:8010 user@gpu_server
```

这样本地 `localhost:8010` 会被转发到远程 GPU 服务器的 `localhost:8010`。

步骤 3：在本地 ORCA 机器运行推理并接入评分

```
conda activate orcalab_lerobot
cd /path/to/SouthGrid/src/examples/inference/g1_omnipicker

# 同样需要先在 eval 脚本中嵌入 ScorerClient
python eval_g1_omnipicker_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost --port 8010 \
    --prompt "按红色按钮" \
    --team-id team_xxx --team-token token_team_xxx \
    --robot-id g1_omnipicker \
    --task-id task2_button_press \
    --targets red green blue yellow
```

此时：

- `eval_g1_omnipicker_lerobot.py` 通过 `localhost:8010` 访问远程策略服务；
- `ScorerClient` 在本地自动启动评分服务，连接本地 ORCA `localhost:50051`；
- 屏幕录制在本地进行，视频上传中央服务器。

替代方案：直接指定远程 IP

如果网络允许直接访问远程 8010 端口（无防火墙），可以在推理脚本中直接指定远程 IP：

```
python eval_g1_omnipicker_lerobot.py \
    --host gpu_server_ip --port 8010 \
    ...
```

但需要注意：

- 确保远程策略服务监听 `0.0.0.0`；
- 确保本地到远程 8010 端口 TCP 可达；
- 大模型权重推理对带宽/延迟敏感，建议优先使用 SSH 隧道或同一局域网。

7.5 模型训练后如何接入评分

训练完成并得到 checkpoint 后，接入评分的步骤如下：

1. **确认推理脚本可以单独跑通**：不启动评分，仅运行 `eval_*.py`，观察机器人是否能完成任务；
2. **在推理脚本中导入 `ScorerClient`**；
3. **在 `env.reset()` 和场景就绪后、控制循环前调用 `start_attempt`**；
4. **在控制循环正常结束或异常退出时调用 `finish`**；
5. **保持进程存活等待视频上传**（默认 60-90 秒）。

一个最小可复用的评分包装函数：

```
from contextlib import contextmanager
from orca_scorer_client import ScorerClient

@contextmanager
def scoring_session(team_id, team_token, robot_id, task_id, targets=None,
prompt=None):
    scorer = ScorerClient(
        team_id=team_id,
        team_token=team_token,
        robot_id=robot_id,
        timeout=60.0,
    )
    info = scorer.start_attempt(task_id, targets=targets, prompt=prompt)
    print(f"[scorer] attempt_id={info.get('attempt_id')}")
    try:
        yield scorer, info
    finally:
        summary = scorer.finish()
        print(f"[scorer] score={summary.get('score')}/{summary.get('max_score')}")
        for step in summary.get("step_results", []):
            print(f"  {step['step_id']}: {'PASS' if step['success'] else 'FAIL'}")
        # 等待视频上传
        import time
        time.sleep(90)
```

使用示例：

```
with scoring_session(
    team_id="team_xxx",
    team_token="token_team_xxx",
    robot_id="g1_omnipicker",
    task_id="task2_button_press",
    targets=["red", "green", "blue", "yellow"],
    prompt="按红色按钮",
):
    run_your_inference_loop()
```

7.6 ScorerClient API 速查

方法	路径	调用时机	说明
`start_attempt(task_id, targets, prompt)`	`POST /api/attempt/start`	控制循环开始前	开始尝试并返回 `attempt_id`

`begin_step(step_id)`	`POST /api/step/begin`	手动步骤模式	开始指定步骤
`end_step()`	`POST /api/step/end`	手动步骤模式	结束当前步骤并自动判定
`set_extra(key, value)`	`POST /api/step/extra`	任意时刻	设置步骤额外元数据
`finish()`	`POST /api/attempt/finish`	控制循环结束后	结束尝试、返回汇总、触发视频上传
`status()`	`GET /api/status`	任意时刻	查询当前评分状态
`history()`	`GET /api/attempts`	任意时刻	查询历史尝试

7.7 配置优先级

`ScorerClient` 按以下优先级读取配置：

1. 代码参数
2. 环境变量 `ORCA_SCORING_*`
3. `orca_scoring.yaml` 配置文件
4. 内置默认值

常用环境变量：

变量	作用	默认值
`ORCA_SCORING_TEAM_ID`	队伍 ID	-
`ORCA_SCORING_TEAM_TOKEN`	队伍 Token	-
`ORCA_SCORING_BASE_URL`	本地评分服务地址	`http://127.0.0.1:9000`
`ORCA_SCORING_SERVER_URL`	中央评分服务器	`http://8.159.130.194/`
`ORCA_SCORING_ORCA_ADDR`	ORCA gRPC 地址	`localhost:50051`
`ORCA_SCORING_ROBOT_ID`	机器人形态	`g1_omnipicker`
`ORCA_SCORING_VIDEO_ENABLED`	是否录屏	`1`

7.8 查看结果

本地汇总文件：

```
.orca_comp_scorer/summary_att_<attempt_id>.json
```

中央服务器：登录中央服务器（`http://8.159.130.194/`），使用各团队收到的 team id 和 token，以开发者模式登录，即可查看本团队的评分结果、任务记录与任务视频。

> **注意**：禁止用户恶意修改评分客户端，伪造数据，以及恶意攻击评分服务器。

第 8 章

常见问题与排查

8.1 环境安装

问题：`pip install pin` 后 NumPy/Pinocchio 冲突 **处理**：删除当前环境，重新按 `environment-unitree.yml` 创建，禁止手动安装 pin/casadi。

问题：`verify_environment.py` 报相机库错误 **处理**：检查 PyAV、FFmpeg 是否支持 `av1_nvenc`，确认 GPU 为 RTX 40 系及以上。

8.2 数据采集

问题：相机超时 **处理**：

1. 确认 OrcaLab 中相机端口与脚本一致；
2. 确认 `UseNvEnc`、`Color Camera`、`Recording` 均已勾选；
3. 三路相机全部配置完后再统一勾选 Recording。

问题：续采后机器人不动 **处理**：Pico 遥操作时，手柄已连接但尚未开始采集，需再按一次左侧握键。

问题：宇树关节名报错 **处理**：确认 `--agent_name` 与场景中机器人名称一致，如 `g1_pick`。

8.3 训练

问题：训练报 HuggingFace Hub 连接错误 **处理**：确认已对 `data_loader.py` 打本地数据集补丁，且数据集位于 `HF_LEROBOT_HOME/<repo_id>/`。

问题：OOM **处理**：

- 降低 `batch_size`（LoRA 从 32 降到 16）
- 降低 `XLA_PYTHON_CLIENT_MEM_FRACTION`
- 缩短 `action_horizon`

8.4 推理与模型控制

问题：WebSocket 连接失败 **处理**：确认策略服务器已启动，防火墙放行 8010 端口，或使用 SSH 隧道：

```
ssh -L 8010:localhost:8010 user@<server_ip>
```

问题：机械臂抖动或跟踪发散 **处理**：

1. 确认训练与推理使用同一套动作约定；
2. 检查是否加载了正确的 checkpoint；
3. 调整 `--kp` 刚度参数；
4. 确认 OSC 控制器的 `frame_skip` 与训练时数据频率一致。

问题：策略输出动作维度与控制器不匹配 **处理**：

- 智元 G1 OmniPicker 需要 18 维动作（双臂末端位姿 + 夹爪）；
- 宇树 G1 需要 28 维关节角动作；
- 检查 `TrainConfig` 中的 `action_dim` 是否与推理脚本一致。

8.5 评分接入

问题：评分服务启动失败 **处理**：

1. 检查端口 9000 是否被占用；
2. 确认评分客户端已通过华为云 Git 仓库安装：

```
pip install git+https://codehub.devcloud.cn-east-
3.huaweicloud.com/bbb453d36204472ca668f7c8ca786e59/competition_scoring_client.gi
t@SouthGrid
```

问题：评分客户端安装后无法正常使用 **处理**：

1. 确认使用华为云 Git 仓库安装：

```
pip install git+https://codehub.devcloud.cn-east-
3.huaweicloud.com/bbb453d36204472ca668f7c8ca786e59/competition_scoring_client.gi
t@SouthGrid
```

2. 不要通过本地源码目录 `pip install -e .` 安装。

问题：中央服务器看不到结果 **处理**：

1. 确认 `ORCA_SCORING_VIDEO_ENABLED=1`；
2. 调用 `finish()` 后继续等待 60-90 秒让视频上传；
3. 登录中央服务器（开发者模式），确认结果是否已上报。

问题：视频录制不完整 **处理**：

1. ORCA 窗口必须在主显示器、置顶；
2. 窗口分辨率不超过 1920×1080；
3. 控制循环加 real-time 节流，保持墙钟与仿真时间一致；
4. 确认 `ScorerClient` 与 ORCA 在同一台机器上运行。

第 9 章

附录：关键文件与命令速查

9.1 关键文件

文件	路径
仓库总览	`SouthGrid/README.md`
智元采集文档	`SouthGrid/docs/g1_omnipicker_collection.md`
智元推理文档	`SouthGrid/docs/g1_omnipicker_inference.md`
宇树采集文档	`SouthGrid/docs/unitree_g1_collection.md`
OpenPI 部署文档	`SouthGrid/docs/openpi_deployment.md`

9.2 常用命令

```
# 激活环境
conda activate orcalab_lerobot

# 验证环境
python scripts/verify_environment.py

# Task 2 智元采集
cd src/examples/dataCollection/g1_omnipicker
python g1_omnipicker_collection_scripted_button_lerobot.py \
    --task_config ../common/example.yaml \
    --lerobot_out ~/datasets/g1_button_scripted \
    --repo_id local/g1_omnipicker_button \
    --counts 25,25,25,25 --fps 20 --clock wall

# Task 3 智元采集
cd src/examples/dataCollection/g1_omnipicker
python g1_omnipicker_collection_scripted_tool_lerobot.py \
    --task_config ../common/example.yaml \
    --lerobot_out ~/datasets/g1_tool_scripted \
    --repo_id local/g1_omnipicker_tool \
    --num_episodes 20 --fps 20

# Task 2 智元推理
cd src/examples/inference/g1_omnipicker
python eval_g1_omnipicker_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost --port 8010 \
    --prompt "按红色按钮" --max_steps 500 --episodes 3

# Task 3 智元推理
cd src/examples/inference/g1_omnipicker
python eval_g1_omnipicker_tool_lerobot.py \
    --task_config ../../dataCollection/common/example.yaml \
    --host localhost --port 8010 \
    --prompt "整理工具" --max_steps 10000 --episodes 1

# OpenPI 计算归一化统计
cd /path/to/openpi
HF_LEROBOT_HOME=/data/lerobot_datasets \
uv run scripts/compute_norm_stats.py --config-name pi05_g1_omnipicker_button_lora

# OpenPI 训练
```

```
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 HF_LEROBOT_HOME=/data/lerobot_datasets \
XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 WANDB_MODE=offline \
uv run scripts/train.py pi05_g1_omnipicker_button_lora --exp-name=exp01 --overwrite

# OpenPI 推理服务
cd /path/to/openpi
CUDA_VISIBLE_DEVICES=0 \
XLA_PYTHON_CLIENT_PREALLOCATE=false \
XLA_PYTHON_CLIENT_ALLOCATOR=platform \
uv run scripts/serve_policy.py --port 8010 policy:checkpoint \
    --policy.config=pi05_g1_omnipicker_button_lora \
    --policy.dir=checkpoints/pi05_g1_omnipicker_button_lora/exp01/9999

# 查询中央服务器结果
# 登录 http://8.159.130.194/ ，使用本团队 team id / token，以开发者模式登录查看
```

第 10 章

写在最后

本指南基于当前仓库实际代码整理，力求客观、完整。需要特别注意以下事实：

1. **本文档中所有代码均为参考样例**，用户可以在不破坏评分规则的前提下自行重新实现控制、训练或评分接入逻辑。
2. **Task 3 目前缺少完整的评分运行脚本。** 开发者需要基于 `eval_g1_omnipicker_tool_lerobot.py` 或自己的控制逻辑，参照 Task 2 的方式自行实现评分接入。
3. **模型训练在独立的 OpenPI 环境中完成**，本仓库只负责数据采集和在线推理客户端。
4. `ScorerClient` **及其本地评分服务必须与 ORCA 仿真运行在同一台物理机上**，策略服务器可以远程部署并通过 SSH 隧道或直联方式访问。
5. **评分系统的稳定性高度依赖外部环境**：ORCA 场景加载、相机端口配置、窗口位置、视频录制、网络上传等任何一环出错都可能导致评分失败，建议逐项验证。
6. **评分客户端必须通过华为云 Git 仓库安装**（`pip install git+https://codehub.devcloud.cn-east-3.huaweicloud.com/bbb453d36204472ca668f7c8ca786e59/competition_scoring_client.git@SouthGrid`），不要通过本地源码目录 `pip install -e .` 安装，否则可能导致评分服务无法自动启动。
7. **Linux 下要实现视频录制，还需额外安装** `pyautogui`、`python-xlib`、`imageio-ffmpeg`、`mss` 等 Python 包，以及系统级 `ffmpeg`、`xdotool`、`wmctrl`；Wayland 用户还需 `grim`、`slurp`。

第 11 章

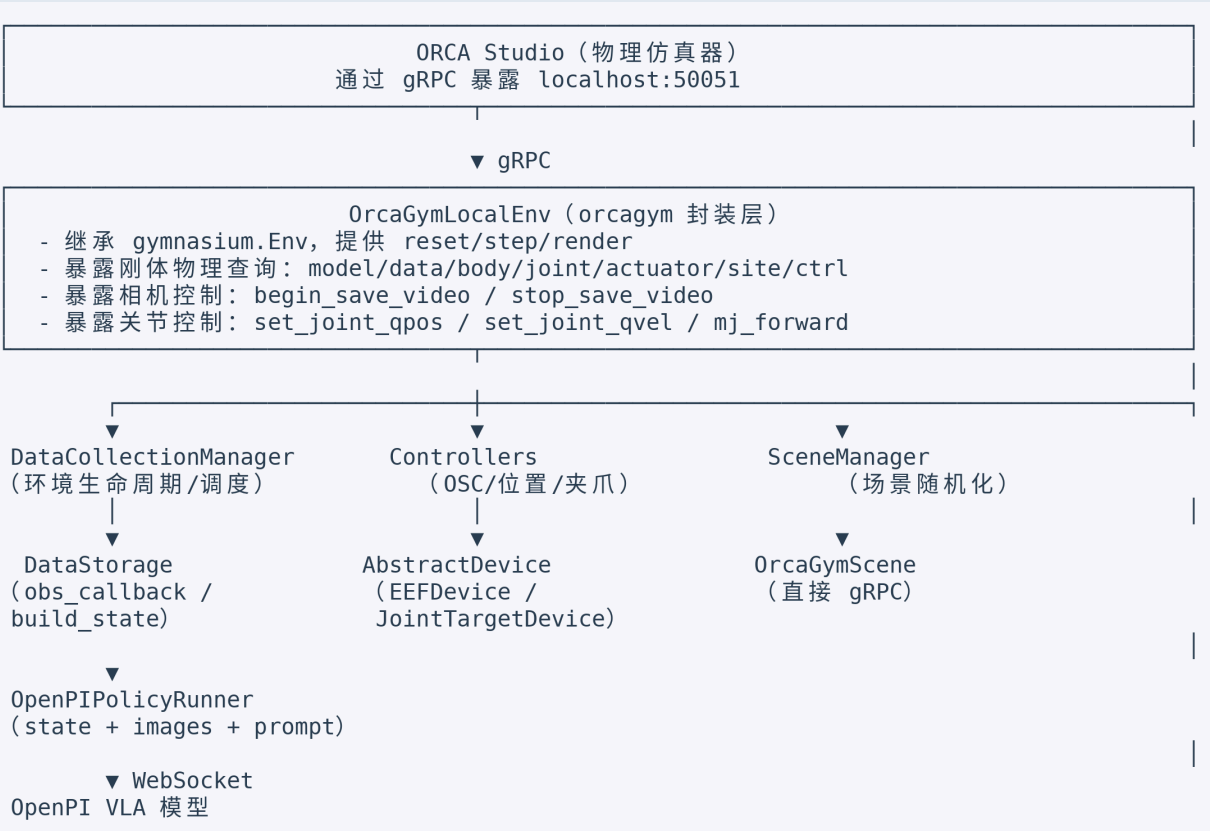
OrcaGym API 完整参考手册

`SouthGrid` 仓库本身不直接操作底层物理模型 XML，而是把 `orcagym` 当作**仿真抽象层 + 控制中间件**来使用。ORCA 支持多物理场后端，MuJoCo 是其刚体求解器之一。本章基于仓库实际调用的源码，给出 `orcagym` 关键 API 的完整参考：函数签名、参数说明、返回值与可直接运行的代码片段。

11.1 模块与文件位置

模块	主要文件	作用
`orca_gym.environment.orca_gym_local_env`	`OrcaGym/orca_gym/environment/orca_gym_local_env.py`	环境生命周期、状态查询、控制指令
`orca_gym.environment.orca_gym_env_mixin`	`OrcaGym/orca_gym/environment/orca_gym_env_mixin.py`	名称空间解析、动作/观测空间生成
`orca_gym.core.orca_gym_model`	`OrcaGym/orca_gym/core/orca_gym_model.py`	刚体模型内省（body/joint/actuator/site 等，基于 MuJoCo 后端）
`orca_gym.core.orca_gym_data`	`OrcaGym/orca_gym/core/orca_gym_data.py`	仿真数据对象封装
`orca_gym.sensor.rgbd_camera`	`OrcaGym/orca_gym/sensor/rgbd_camera.py`	WebSocket 相机客户端
`orca_gym.scene.orca_gym_scene`	`OrcaGym/orca_gym/scene/orca_gym_scene.py`	gRPC 场景管理
`orca_gym.adapters.robosuite.controllers`	`OrcaGym/orca_gym/adapters/robosuite/controllers/*`	Robosuite 控制器工厂与基类
`orca_gym.devices.pico_joytsick`	`OrcaGym/orca_gym/devices/pico_joytsick.py`	Pico VR 遥操作手柄
`orca_gym.log.orca_log`	`OrcaGym/orca_gym/log/orca_log.py`	统一日志

11.2 整体关系图



11.3 环境生命周期 API（OrcaGymLocalEnv）

11.3.1 构造函数

```
class OrcaGymLocalEnv(OrcaGymBaseEnv):
    def __init__(
        self,
        frame_skip: int,
        orcagym_addr: str,
        agent_names: list[str],
        time_step: float,
        **kwargs,
    ):
        ...
```

参数	类型	必填	说明

`frame_skip`	`int`	是	每调用一次 `step()`，ORCA 内部推进的物理步数
`orcagym_addr`	`str`	是	ORCA gRPC 地址，如 `localhost:50051`
`agent_names`	`list[str]`	是	智能体名称列表，决定 `env.body()` 等前缀
`time_step`	`float`	是	单物理步时长，如 `0.001`
`default_joint_values`	`dict`	否	`reset()` 时恢复的初始关节值
`obs_callback`	`Callable[[OrcaGymLocalEnv], dict]`	否	每步调用以构造原始观测

仓库中通常不直接实例化 `OrcaGymLocalEnv`，而是通过 `DataCollectionManager.create_env()` 注册并创建：

```
# SouthGrid/src/dataCollectionManager/data_collection_manager.py
import gymnasium as gym

env_id = f"{env_name}-OrcaGym-{orcagym_addr.replace(':', '-')}-{env_index:03d}"
gym.register(
    id=env_id,
    entry_point="envs.dataCollection.dataCollection_env:DataCollectionEnv",
    kwargs={
        "frame_skip": frame_skip,
        "orcagym_addr": orcagym_addr,
        "agent_names": [agent_name],
        "time_step": time_step,
        "default_joint_values": default_joint_values,
        "obs_callback": obs_callback,
    },
)
env = gym.make(env_id, **kwargs).unwrapped
```

11.3.2 核心交互

```
# 标准 Gymnasium 接口
obs, reward, terminated, truncated, info = env.step(action)
obs, info = env.reset(seed=None, options=None)
env.render()
env.close()
```

方法	签名	作用	注意
`step`	`step(action) -> tuple[ObsType, SupportsFloat, bool, bool, dict]`	将执行器命令发给 ORCA，推进 `frame_skip` 个物理步	`action` 必须是完整 `model.nu` 长度，缺失维度会报错
`reset`	`reset(seed=None, options=None) -> tuple[ObsType, dict]`	重置场景到初始状态	先 `reset_simulation()`，再调用子类 `reset_model()`；会恢复 `default_joint_values`
`render`	`render()`	同步触发渲染/可视化	不带 `--no-preview` 的推理脚本会频繁调用
`close`	`close()`	关闭环境并清理 gRPC 通道	程序退出前建议调用

11.3.3 仿真推进与同步

```
# 一次性推进 n_frames 个物理步，并在内部设置控制
env.do_simulation(ctrl, n_frames) -> None

# 仅设置执行器控制数组
env.set_ctrl(ctrl) -> None

# 仅推进 nstep 个物理步
env.mj_step(nstep: int) -> None

# 执行刚体前向运动学计算：更新 xpos/xquat/xmat 等动力学状态
env.mj_forward() -> None

# 从 ORCA 服务端同步最新 qpos/qvel/qacc 到本地 env.data
env.update_data() -> None

# 重置仿真到初始帧并同步数据
env.reset_simulation() -> None

# 保存当前状态为 init_qpos / init_qvel，供 reset 使用
env.init_qpos_qvel() -> None

# 修改 time_step，并同步更新 realtime_step
env.set_time_step(time_step: float) -> None
```

11.3.4 常用属性

属性	类型	说明
`env.model`	`OrcaGymModel`	静态刚体模型结构
`env.data`	`OrcaGymData`	动态仿真数据（qpos、qvel、time 等）
`env.ctrl`	`np.ndarray`	长度为 `env.model.nu` 的执行器控制数组
`env.nu` / `env.nv` / `env.nq`	`int`	执行器、自由度、位置维度
`env.frame_skip`	`int`	每次 `step()` 推进的物理步数
`env.time_step`	`float`	单物理步时长

`env.realtime_step`	`float`	`time_step * frame_skip`，即每次 `step()` 对应的仿真时间

11.4 模型内省 API（OrcaGymModel）

`env.model` 是查询刚体模型结构的入口，提供 body / joint / actuator / site / geom 的名称↔ID 映射。

11.4.1 Body

```
env.model.body_name2id(body_name: str) -> int
env.model.body_id2name(body_id: int) -> str
env.model.get_body_dict() -> dict[str, dict]
env.model.get_body_byname(name: str) -> dict
env.model.get_body_byid(id: int) -> dict
env.model.get_body_names() -> KeysView[str]
```

11.4.2 Joint

```
env.model.joint_name2id(joint_name: str) -> int
env.model.joint_id2name(joint_id: int) -> str
env.model.get_joint_dict() -> dict[str, dict]
env.model.get_joint_byname(name: str) -> dict
env.model.get_joint_byid(id: int) -> dict
env.model.get_joint_qposrange(joint_names: list) -> np.ndarray  # shape (N, 2)
```

11.4.3 Actuator

```
env.model.actuator_name2id(actuator_name: str) -> int
env.model.actuator_id2name(actuator_id: int) -> str
env.model.get_actuator_dict() -> dict[str, dict]
env.model.get_actuator_byname(name: str) -> dict
env.model.get_actuator_byid(id: int) -> dict
env.model.get_actuator_ctrlrange() -> np.ndarray  # shape (nu, 2)
```

11.4.4 Site / Geom

```
# Site
env.model.site_name2id(site_name: str) -> int
env.model.site_id2name(site_id: int) -> str
env.model.get_site_dict() -> dict[str, dict]
env.model.get_site(name_or_id: Union[str, int]) -> dict

# Geom
env.model.geom_name2id(geom_name: str) -> int
env.model.geom_id2name(geom_id: int) -> str
env.model.get_geom_dict() -> dict[str, dict]
env.model.get_geom_byname(name: str) -> dict
env.model.get_geom_byid(id: int) -> dict
env.model.get_geom_body_name(geom_id: int) -> str
env.model.get_geom_body_id(geom_id: int) -> int
```

11.4.5 使用示例

```
# 获取执行器控制范围并生成动作空间
ctrlrange = env.model.get_actuator_ctrlrange()  # shape (nu, 2)
action_space = env.generate_action_space(ctrlrange)

# 获取所有 body 名称
for body_name in env.model.get_body_names():
    if "base_link" in body_name:
        base_body = body_name
        break

# 名称→ID 映射，用于构造 env.ctrl 索引
arm_actuator_ids = [
    env.model.actuator_name2id(env.actuator(name))
    for name in arm_motor_names
]
```

11.5 名称空间解析 API（OrcaGymEnvMixin）

`OrcaGymLocalEnv` 通过 mixin 提供自动加 agent 前缀的方法，避免硬编码完整 XML 名。

```
env.body(name: str, agent_id: int = None) -> str
env.joint(name: str, agent_id: int = None) -> str
env.actuator(name: str, agent_id: int = None) -> str
env.site(name: str, agent_id: int = None) -> str
env.mocap(name: str, agent_id: int = None) -> str
env.sensor(name: str, agent_id: int = None) -> str
```

- `agent_id=None` 时使用 `_agent_names[0]` 作为前缀。
- 例如 `agent_names=["g1_omnipicker"]` 时，`env.body("base_link")` 返回 `"g1_omnipicker_base_link"`。

```
# 示例：构造完整执行器名
full_actuator = env.actuator("shoulder_roll_r")
print(full_actuator)  # g1_omnipicker_shoulder_roll_r

# 多智能体场景可指定 agent_id
full_site = env.site("ee_site_right", agent_id=1)
```

11.6 状态查询 API

11.6.1 关节状态

```
qpos = env.query_joint_qpos(joint_names: list[str]) -> dict[str, Any]
qvel = env.query_joint_qvel(joint_names: list[str]) -> dict[str, Any]
qacc = env.query_joint_qacc(joint_names: list[str]) -> dict[str, Any]
```

返回值以关节名为键。单个自由关节的值是长度为 7 的数组，普通旋转关节为标量或长度为 1 的数组。

11.6.2 关节地址与偏移

```
qpos_offsets, qvel_offsets, qacc_offsets = env.query_joint_offsets(joint_names)
qpos_lengths, qvel_lengths, qacc_lengths = env.query_joint_lengths(joint_names)

qpos_adr = env.jnt_qposadr(joint_name: str) -> int
dof_adr = env.jnt_dofadr(joint_name: str) -> int
```

`jnt_qposadr` / `jnt_dofadr` 分别返回关节在全局 `data.qpos` / `data.qvel` 数组中的起始索引，常用于直接切片底层数组。

```
joint = env.model.get_joint_byname("waist_yaw")
nq = joint["JointNq"]
adr = env.jnt_qposadr("waist_yaw")
values = env.data.qpos[adr:adr + nq]
```

11.6.3 Body 位姿

```
xpos, xmat, xquat = env.get_body_xpos_xmat_xquat(body_name_list: list[str])
# xpos:  shape (N * 3,)
# xmat:  shape (N * 9,)，按行展开的 3x3 旋转矩阵
# xquat: shape (N * 4,)，格式 [w, x, y, z]

xpos, xmat, xquat, xvel = env.get_body_xpos_xmat_xquat_xvel(body_name_list)
# xvel: shape (N, 3)，COM 线速度
```

11.6.4 Site 位姿

```
# 世界坐标系：位置和旋转矩阵
site_dict = env.query_site_pos_and_mat(site_names: list[str])
# {name: {"xpos": np.ndarray(3,), "xmat": np.ndarray(9,)}}

# 世界坐标系：位置和四元数 [w, x, y, z]
site_dict = env.query_site_pos_and_quat(site_names: list[str])
# {name: {"xpos": np.ndarray(3,), "xquat": np.ndarray(4,)}}

# 相对 base_body 的 body 坐标系
site_dict = env.query_site_pos_and_quat_B(
    site_names: list[str],
    base_body_list: list[str],
)
# {name: {"xpos": np.ndarray(3,), "xquat": np.ndarray(4,)}}
```

方法	返回坐标系	典型用途
`query_site_pos_and_quat`	世界坐标系	通用位姿查询、计算距离
`query_site_pos_and_quat_B`	相对 `base_body` 的局部坐标系	策略输入、OSC 目标、评分判定

11.6.5 Site 速度

```
xvalp, xvalr = env.query_site_xvalp_xvalr(site_names: list[str])
# 世界坐标系线速度 / 角速度

xvalp_B, xvalr_B = env.query_site_xvalp_xvalr_B(
    site_names: list[str],
    base_body_list: list[str],
)
# 相对 base_body 的线速度 / 角速度
```

11.6.6 其他常用查询

```
sensor_data = env.query_sensor_data(sensor_names: list[str]) -> dict
torques = env.query_actuator_torques(actuator_names: list[str]) -> dict
site_size = env.query_site_size(site_names: list[str]) -> dict

# Body 相对 base 的速度/位置/姿态（固定基座场景常用）
vel_B = env.query_velocity_body_B(ee_body: str, base_body: str)
pos_B = env.query_position_body_B(ee_body: str, base_body: str)
ori_B = env.query_orientation_body_B(ee_body: str, base_body: str)
```

11.7 控制指令 API

11.7.1 直接设置关节

```
env.set_joint_qpos(joint_qpos: dict) -> None
env.set_joint_qvel(joint_qvel: dict) -> None
env.mj_forward()  # 设置后必须调用，才能更新动力学状态
```

`joint_qpos` / `joint_qvel` 的字典键为关节名，值为标量或数组。常用于场景随机化、重置机器人姿态、Task 1 的浮动关节位姿积分。

```
# 示例：把机器人整体移动到一个新位置
free_joint = env.joint("floating_base")
env.set_joint_qpos({free_joint: [x, y, z, qw, qx, qy, qz]})
env.set_joint_qvel({free_joint: [0.0] * 6})
env.mj_forward()
```

11.7.2 执行器控制

```
# 直接设置完整执行器数组
env.set_ctrl(ctrl: Union[np.ndarray, dict]) -> None

# 或通过 env.ctrl 数组下标读写
env.ctrl[actuator_id] = value
```

控制器通常返回 `{actuator_id: value}` 字典，`DataCollectionManager.run_controllers()` 会汇总到 `self.ctrl` 数组，再交给 `env.step(action)`。

11.7.3 禁用执行器组

```
# 直接调用环境方法
env.disable_actuator(actuator_groups: list[int]) -> None

# 或通过 DataCollectionManager 包装
manager.set_disable_actuator_group([agent_conf.positions_group])
```

智元 OmniPicker 推理时必须禁用位置执行器组，否则 OSC 控制器与位置执行器会冲突。

11.7.4 仿真步进汇总

方法	作用	使用场景
`env.step(action)`	完整控制循环：设控制 + 推进 `frame_skip` 步 + 同步数据	每控制周期调用
`env.do_simulation(ctrl, n_frames)`	设置控制并推进 `n_frames` 步	需要自定义步进步数
`env.mj_step(nstep)`	只推进，不设置控制	已有 `set_ctrl` 后使用
`env.mj_forward()`	前向计算，更新 xpos/xquat	手动改 qpos 后必须调用

11.8 相机与视频 API

11.8.1 触发/停止推流

```
from orca_gym.core.orca_gym_local import CaptureMode

env.begin_save_video(
    file_path: str,
    capture_mode: CaptureMode = CaptureMode.ASYNC,
) -> None

env.stop_save_video() -> None
```

- `file_path` 是目录或文件前缀，ORCA 用它决定视频保存位置并触发 WebSocket 推流。
- **必须先调用** **`begin_save_video`**，`CameraWrapper` 才能取到帧；`CameraWrapper` 订阅的是 ORCA 推流，不是直接抓屏。

11.8.2 CameraWrapper

```
from orca_gym.sensor.rgbd_camera import CameraWrapper

cam = CameraWrapper(name="camera_head_color", port=7090)
cam.start()                                    # 在后台线程中连接 ws://localhost:7090
frame, image_index = cam.get_frame(
    format="rgb24",                            # "bgr24"（默认）或 "rgb24"
    size: tuple = None,                        # 可选 (W, H) 缩放目标
)
cam.stop()                                     # 停止后台线程
```

- `frame`：`np.ndarray(shape=(H, W, 3), dtype=np.uint8)`。
- `image_index`：帧序号，用于诊断掉帧或重复帧。

11.8.3 仓库封装工具

```
from dataStorage.lerobot_camera import (
    omnipicker_camera_map,
    bring_up_cameras,
    capture_frame_with_idx,
    probe_camera_hw,
    close_cameras,
)

# 默认 {env_sensor_name: (lerobot_key, port)}
camera_map = omnipicker_camera_map(enable_wrist_l=False)
# 带 TCP 端口探测、首帧等待、线程重启的稳健启动
cameras = bring_up_cameras(camera_map, port_timeout=30.0, frame_timeout=30.0)

# 从首帧探测真实分辨率 (H, W)
target_hw = probe_camera_hw(cameras, camera_map)  # e.g. (480, 640)

# 每步取最新帧并缩放到目标分辨率
images, indices = capture_frame_with_idx(cameras, camera_map, target_hw)
# images: {lerobot_key: np.ndarray(H, W, 3) RGB uint8}

# 清理
close_cameras(cameras)
```

11.9 场景管理 API

11.9.1 OrcaGymScene（gRPC 直连 ORCA）

```
from orca_gym.scene.orca_gym_scene import OrcaGymScene

scene = OrcaGymScene(grpc_addr="localhost:50051")

scene.initialize_grpc()
scene.publish_scene()
scene.add_actor(actor)
scene.set_light_info(actor_name: str, light_info: LightInfo)
scene.set_camera_sensor_info(actor_name: str, camera_sensor_info: CameraSensorInfo)
scene.make_camera_viewport_active(actor_name: str, entity_name: str)
scene.set_material_info(actor_name: str, material_info: MaterialInfo)
scene.set_actor_anim_param_number(actor_name: str, param_name: str, value: float)
scene.set_actor_anim_param_bool(actor_name: str, param_name: str, value: bool)
scene.set_actor_anim_param_string(actor_name: str, param_name: str, value: str)
scene.set_actor_lua_param_string(actor_name: str, param_name: str, value: str)
scene.set_image_enabled(actor_name: str, enabled: bool)
scene.get_rundata(scriptname: str, stepname: str)
scene.close()
```

11.9.2 SceneManager（仓库封装）

```
from scene.scene_manager import SceneManager

manager = SceneManager(grpc_addr="localhost:50051", config=config)
manager.set_env(env)

# 设置/读取 actor 关节位置
manager.set_actor_qpos(joint_name: str, qpos: np.ndarray) -> None
qpos = manager.get_actor_qpos(joint_name: str) -> np.ndarray

# 场景生成与更新
manager.spawn_scene()
manager.update_scene()
```

11.10 控制器适配 API

11.10.1 加载配置与创建控制器

```
from orca_gym.adapters.robosuite.controllers import (
    controller_factory,
    load_controller_config,
)

# 加载默认控制器配置
config = load_controller_config(default_controller="OSC_POSE")
# 可选: "JOINT_POSITION", "JOINT_TORQUE", "JOINT_VELOCITY",
#       "OSC_POSITION", "OSC_POSE", "IK_POSE"

# 使用配置创建控制器实例
controller = controller_factory(name="OSC_POSE", params=params)
```

`params` 中通常需要包含 `sim`（即 `env.gym`）、`eef_name`、关节索引、执行器范围、控制频率等。

11.10.2 控制器基类

```
from orca_gym.adapters.robosuite.controllers.base_controller import Controller

class MyController(Controller):
    def run_controller(self):
        # 将当前目标 action 转换为执行器指令
        ...
        return {actuator_id: value, ...}
```

常用方法：

方法	签名	作用
`run_controller`	`run_controller() -> dict[int, float]`	生成执行器控制指令
`update`	`update(force=False)`	更新内部状态（ee_pos、jacobian、mass matrix 等）
`scale_action`	`scale_action(action)`	将输入动作裁剪并缩放到输出范围
`reset_goal`	`reset_goal()`	重置目标

11.11 输入设备 API

```
from orca_gym.devices.pico_joytsick import PicoJoystick, PicoJoystickKey

# 实时模式（Pico 手柄通过 adb 转发到此端口）
joy = PicoJoystick(port=8001)

# 回放模式（用录制好的 JSON 数据回放）
joy = PicoJoystick(
    replay_mode=True,
    replay_data=PicoJoystick.load_replay_data("recorded_teleop.json"),
)

# 绑定按键事件
joy.bind_key_event("A", lambda transform, key_state: print("A pressed"))

# 获取当前状态
key_state = joy.get_key_state()          # dict
transforms = joy.get_transform_list()    # [left_hand, right_hand, motion_trackers]

# 在主循环中触发事件
joy.update([PicoJoystickKey.A, PicoJoystickKey.R_TRIGGER])

joy.close()
```

`PicoJoystickKey` 枚举包含：X/Y/A/B 、L/R_TRIGGER 、L/R_GRIPBUTTON 、L/R_JOYSTICK_POSITION/PRESSED、L/R_TRANSFORM 等。

11.12 日志 API

```
from orca_gym.log.orca_log import get_orca_logger

logger = get_orca_logger(
    name="Eval",
    log_file="eval.log",
    console_level="INFO",
    file_level="DEBUG",
)

logger.debug("debug")
logger.info("info")
logger.warning("warn")
logger.error("error")
logger.fatal("fatal")
logger.performance("perf")

logger.set_console_level("DEBUG")
```

`get_orca_logger` 返回单例 `OrcaLog`，首次初始化后再次调用会返回同一实例。

11.13 各模块 API 使用统计

模块	主要使用的 orcagym API	作用
`dataCollectionManager/data_collection_manager.py`	`gym.make`, `env.reset`, `env.step`, `env.render`, `set_disable_actuator_group`, `model.get_joint_dict`	环境生命周期、控制器调度
`envs/dataCollection/dataCollection_env.py`	继承 `OrcaGymLocalEnv`, `set_default_joint_values`	自定义 Gym Env
`controllers/controller_arm.py`	`query_site_pos_and_quat_B`, `get_body_xpos_xmat_xquat`, `query_joint_offsets`, `model.actuator_name2id`	OSC 臂控
`controllers/controllers.py`	`env.joint`, `env.actuator`, `query_joint_offsets`, `controller_factory`	创建各类控制器

`dataStorage/*_data_storage.py`	`query_joint_qpos`, `query_site_pos_and_quat_B`, `env.body`, `env.joint`, `env.actuator`, `model.actuator_name2id`, `env.ctrl`, `env.data.time`	观测构造
`dataStorage/lerobot_camera.py`	`env.begin_save_video`, `env.stop_save_video`, `orca_gym.sensor.rgbd_camera.CameraWrapper`	相机推流与取帧
`scene/scene_manager.py`	`OrcaGymScene`, `OrcaGymLocalEnv`, `env.set_joint_qpos`	场景随机化
`devices/abstract_device.py`	`PicoJoystick`, `PicoJoystickKey`	遥操作设备

11.14 与 OpenPI / 评分系统的关系

```
OrcaGym API
    │
    ├── 提供 observation.state（关节/末端位姿/夹爪）
    ├── 提供 observation.images（通过 CameraWrapper 取视频流）
    │
    ▼
OpenPIPolicyRunner.build_observation({state, images, prompt})
    │
    ▼
WebSocket → OpenPI VLA 模型
    │
    ▼
返回 actions → Controller → env.step(action) → ORCA 仿真
    │
    ▼
ScorerClient 读取 ORCA 中 site/body 状态判定得分
```

11.15 使用注意事项

1. **不要绕过 `OrcaGymLocalEnv` 直接修改底层仿真数据**。所有控制应通过 `env.step(action)` 或 `env.set_joint_qpos()` + `env.mj_forward()` 进行，否则 gRPC 状态可能不同步。
2. **尽量使用 `env.body()` / `env.joint()` / `env.actuator()` 辅助方法**。它们会自动加 agent 前缀，避免硬编码完整 XML 名。
3. **相机必须先 `begin_save_video` 才能取到流**。`CameraWrapper` 订阅的是 ORCA 推流，不是直接从渲染器抓屏。
4. **`query_site_pos_and_quat_B` 的 `base_body_list` 必须是 body 名数组**。策略输入和评分判定中通常以机器人 `base_body` 为参考系。
5. **`env.ctrl` 是执行器指令数组**。读取它可以得到当前下发的电机值，但写入应通过 controller + `env.step(action)`。

6. `set_disable_actuator_group` **在智元推理中必须调用**。如果不禁用位置执行器组，OSC 控制会和位置执行器冲突。
7. `OrcaGymScene` **直接走 gRPC**，适合做场景随机化；而 `OrcaGymLocalEnv` 适合每步控制循环。两者不要混用处理同一件事。
8. `mj_forward()` **不会推进仿真时间**，只更新依赖 qpos 的运动学/动力学状态；改完 qpos 后必须调用。
9. `query_joint_qpos` **返回的字典值可能是标量或数组**。拼接为向量前注意检查类型与维度。
10. **使用** `bring_up_cameras` **而非裸** `CameraWrapper`。它内置 TCP 端口探测、首帧等待和线程重启，能避免端口未就绪导致的连接失败。

第 12 章

相机配置 → 取帧 → OBS 构造 → VLA 推理 完整实践

本章把第 6 章和第 11 章中的相机/推理细节串联成一条可复现、可复用的流水线。代码主要参考 `SouthGrid/src/examples/inference/g1_omnipicker/eval_g1_omnipicker_lerobot.py`。

12.1 总体链路

```
ORCA Studio 场景加载并渲染
                                                                    |
        ▼
[1] 配置相机组件 + 端口 + UseNvEnc/Color Camera/Recording
                                                                    |
        ▼
[2] env.begin_save_video(trigger_path) 触发 WebSocket 推流
                                                                    |
        ▼
[3] bring_up_cameras(camera_map) 探测端口、连接 CameraWrapper、等首帧
                                                                    |
        ▼
[4] 每控制周期 capture_frame_with_idx / CameraObservationBuilder.build_images
                                                                    |
        ▼
[5] OpenPIPolicyRunner.build_observation({state, images, prompt})
                                                                    |
        ▼
[6] policy.infer(observation) → action_chunk
                                                                    |
        ▼
[7] parse_policy_action → set_target → manager.run_controllers → env.step
                                                                    |
        ▼
ScorerClient 评分 / 视频上传
```

12.2 ORCA Studio 相机配置

以智元 G1 OmniPicker 为例，场景内需包含以下 Camera 组件：

相机	OrcaLab 传感器名	Color Port	用途
头部	`camera_head_color`	7090	全局视野
右腕	`camera_wrist_r_color`	7080	手部操作近景
左腕	`camera_wrist_l_color`	7070	可选（旧三路策略）

每个 Camera 组件必须勾选：

- **UseNvEnc**
- **Color Camera**
- **Recording**

> **关键顺序**：先把三路相机全部添加到场景并配置好端口，再统一勾选 `Recording` 或运行脚本。逐个勾选容易导致端口启动顺序不一致。

12.3 触发相机推流

```
import os

STREAM_TRIGGER_PATH = "/tmp/eval_g1_lerobot_stream"
os.makedirs(STREAM_TRIGGER_PATH, exist_ok=True)

# 必须在场景渲染后开始
env.begin_save_video(STREAM_TRIGGER_PATH)
```

`begin_save_video` 会通知 ORCA Studio 在配置的端口上开始 WebSocket 视频推流。录制文件实际保存路径为 `{STREAM_TRIGGER_PATH}/video/{camera_sensor_name}.mp4`，因此 `STREAM_TRIGGER_PATH` 目录必须预先存在且可写。

12.4 连接 WebSocket 取帧

```
from dataStorage.lerobot_camera import (
    omnipicker_camera_map,
    bring_up_cameras,
    probe_camera_hw,
    close_cameras,
)

# camera_map: {env_sensor_name: (lerobot_key, port)}
camera_map = omnipicker_camera_map(enable_wrist_l=False)

# 探测端口、连接、等首帧，失败自动跳过并打印 WARNING
cameras = bring_up_cameras(camera_map, port_timeout=30.0, frame_timeout=30.0)

# 从首帧探测真实分辨率 (H, W)，失败回退 (480, 640)
target_hw = probe_camera_hw(cameras, camera_map)
print(f"相机就绪: {list(cameras.keys())}, 分辨率={target_hw}")
```

`bring_up_cameras` 做的事：

1. 用 TCP `connect_ex` 探测每个端口是否已监听；
2. 端口就绪后再创建 `CameraWrapper` 并 `start()`；
3. 等待首帧到达，若后台线程意外退出则自动重启（最多 3 次）。

> **关于 `image_index`**：`capture_frame_with_idx` 返回的 `indices` 是 `CameraWrapper` 内部计数器。真实 ORCA 流中每收到一帧该计数器会递增；如果连续两次取到相同 index，说明该相机没有新帧到达（可能是推流卡顿或相机未渲染）。离线 mock 流或极端情况下 index 可能保持不变，应结合 FPS 和图像内容综合判断。

12.5 帧预处理与分辨率对齐

```
from dataStorage.lerobot_camera import capture_frame_with_idx

images, indices = capture_frame_with_idx(cameras, camera_map, target_hw)

# images: {lerobot_key: np.ndarray(H, W, 3) RGB uint8}
# indices: {env_sensor_name: image_index}
```

`capture_frame_with_idx` 内部逻辑：

1. 对每个相机调用 `cam.get_frame(format="rgb24")`；
2. 若 `frame.shape != target_hw`，用 `cv2.INTER_AREA` 缩放到 `(W, H)`；
3. 返回 `np.ascontiguousarray` 的 RGB 图。

在推理脚本中，上述预处理被封装在 `CameraObservationBuilder` 里，并把 `HWC` 转置为 `CHW`：

```
import cv2
import numpy as np

class CameraObservationBuilder:
    def __init__(self, cameras, camera_name_map, target_hw=(480, 640)):
        self.cameras = cameras
        self.camera_name_map = camera_name_map   # {env_name: policy_name}
        self.target_hw = target_hw

    def build_images(self) -> dict:
        H, W = self.target_hw
        images = {}
        for env_name, policy_name in self.camera_name_map.items():
            cam = self.cameras.get(env_name)
            if cam is None:
                rgb = np.zeros((H, W, 3), dtype=np.uint8)
            else:
                try:
                    frame, _ = cam.get_frame(format="rgb24")
                    if frame is None or frame.size == 0:
                        rgb = np.zeros((H, W, 3), dtype=np.uint8)
                    else:
                        if frame.shape[0] != H or frame.shape[1] != W:
                            frame = cv2.resize(frame, (W, H),
interpolation=cv2.INTER_AREA)
                        rgb = np.ascontiguousarray(frame, dtype=np.uint8)
                except Exception:
                    rgb = np.zeros((H, W, 3), dtype=np.uint8)
            images[policy_name] = np.transpose(rgb, (2, 0, 1))  # CHW
        return images
```

12.6 构造 OBS

```
import numpy as np

# state 由 dataStorage 的 obs_callback + build_state 构造
state = storage.build_state(storage.obs_callback(env))  # e.g. shape (18,)

# 构造与训练时一致的观测字典
observation = {
    "state": state,                                              # np.ndarray float32
    "images": CameraObservationBuilder(cameras, camera_name_map,
target_hw).build_images(),
    "prompt": "按红色按钮",                                        # 必须与训练时一致
}
```

`camera_name_map` 的键必须与训练数据集 `features` 中的相机键完全一致，例如：

```
camera_name_map = {
    "camera_head_color": "cam_head",
    "camera_wrist_r_color": "cam_wrist_r",
}
```

12.7 VLA 推理

```
from openpi_client import websocket_client_policy
import numpy as np

policy = websocket_client_policy.WebsocketClientPolicy(host="localhost", port=8010)
metadata = policy.get_server_metadata()   # 可用来校验模型配置

result = policy.infer(observation)
actions = np.asarray(result["actions"], dtype=np.float32)

if actions.ndim == 1:
    actions = actions.reshape(1, -1)

# actions shape: (action_horizon, action_dim)
print(actions.shape)  # e.g. (50, 18)
```

推理请求内部使用 `msgpack + numpy` 编码，通过 WebSocket 发送到 OpenPI 策略服务。

12.8 动作解析与执行

```
import numpy as np

_L_GRIP_RANGES = agent_conf.gripper_l["actuator_ranges"]
_R_GRIP_RANGES = agent_conf.gripper_r["actuator_ranges"]


def _denorm_grip(norm_val: float, grip_range: tuple[float, float]) -> float:
    lo, hi = float(grip_range[0]), float(grip_range[1])
    return float(np.clip(norm_val, 0.0, 1.0)) * (hi - lo) + lo


def parse_policy_action(raw_action: np.ndarray) -> dict:
    action = np.asarray(raw_action, dtype=np.float32).reshape(-1)
    if action.size < 18:
        raise ValueError(f"Expected at least 18 action dims, got {action.size}")
    return {
        "l_pos_b": action[0:3],
        "l_quat_b": action[3:7],
        "r_pos_b": action[7:10],
        "r_quat_b": action[10:14],
        "l_grip_inner_norm": float(np.clip(action[14], 0.0, 1.0)),
        "l_grip_outer_norm": float(np.clip(action[15], 0.0, 1.0)),
        "r_grip_inner_norm": float(np.clip(action[16], 0.0, 1.0)),
        "r_grip_outer_norm": float(np.clip(action[17], 0.0, 1.0)),
    }


def action_dict_for_apply(action_dict: dict) -> dict:
    l_inner = _denorm_grip(action_dict["l_grip_inner_norm"], _L_GRIP_RANGES[0])
    l_outer = _denorm_grip(action_dict["l_grip_outer_norm"], _L_GRIP_RANGES[1])
    r_inner = _denorm_grip(action_dict["r_grip_inner_norm"], _R_GRIP_RANGES[0])
    r_outer = _denorm_grip(action_dict["r_grip_outer_norm"], _R_GRIP_RANGES[1])
    return {
        "l_pos_b": np.asarray(action_dict["l_pos_b"], dtype=np.float32),
        "l_quat_b": np.asarray(action_dict["l_quat_b"], dtype=np.float32),
        "r_pos_b": np.asarray(action_dict["r_pos_b"], dtype=np.float32),
        "r_quat_b": np.asarray(action_dict["r_quat_b"], dtype=np.float32),
        "l_grip_ctrl": np.array([l_inner, l_outer], dtype=np.float32),
        "r_grip_ctrl": np.array([r_inner, r_outer], dtype=np.float32),
    }
```

```
# 主循环
device = EEFDevice(l_arm=l_arm, r_arm=r_arm, l_grip=l_grip, r_grip=r_grip)
manager.set_device(device)

for model_action in action_chunk:
    parsed = parse_policy_action(model_action)
    device.set_target(**action_dict_for_apply(parsed))

    for _ in range(args.action_repeat):
        action = manager.run_controllers()
        env.step(action)
        env.render()
```

12.9 可复用 Python 类模板

下面是一个最小可复用的 **相机观测 + 策略推理** 包装类，可直接嵌入自定义推理脚本：

```
import cv2
import numpy as np
from openpi_client import websocket_client_policy
from dataStorage.lerobot_camera import (
    omnipicker_camera_map,
    bring_up_cameras,
    probe_camera_hw,
    close_cameras,
)
from orca_gym.sensor.rgbd_camera import CameraWrapper


class CameraObservationBuilder:
    def __init__(self, cameras: dict[str, CameraWrapper],
                 camera_name_map: dict[str, str], target_hw: tuple = (480, 640)):
        self.cameras = cameras
        self.camera_name_map = camera_name_map
        self.target_hw = target_hw

    def build_images(self) -> dict[str, np.ndarray]:
        H, W = self.target_hw
        images = {}
        for env_name, policy_name in self.camera_name_map.items():
            cam = self.cameras.get(env_name)
            rgb = np.zeros((H, W, 3), dtype=np.uint8)
            if cam is not None:
                try:
                    frame, _ = cam.get_frame(format="rgb24")
                    if frame is not None and frame.size > 0:
                        if frame.shape[0] != H or frame.shape[1] != W:
                            frame = cv2.resize(frame, (W, H),
interpolation=cv2.INTER_AREA)
                        rgb = np.ascontiguousarray(frame, dtype=np.uint8)
                except Exception:
                    pass
            images[policy_name] = np.transpose(rgb, (2, 0, 1))
        return images


class OpenPIPolicyRunner:
    def __init__(
        self,
        host: str,
```

```
        port: int,
        prompt: str,
        camera_name_map: dict[str, str],
        cameras: dict,
        target_hw: tuple = (480, 640),
        use_images: bool = True,
    ):
        self.policy = websocket_client_policy.WebsocketClientPolicy(host=host,
port=port)
        self.metadata = self.policy.get_server_metadata()
        self.prompt = prompt
        self.use_images = use_images
        self.cam_builder = (
            CameraObservationBuilder(cameras, camera_name_map, target_hw)
            if use_images else None
        )

    def build_observation(self, state: np.ndarray) -> dict:
        images = self.cam_builder.build_images() if self.use_images else {}
        return {"state": state, "images": images, "prompt": self.prompt}

    def infer_action_chunk(self, state: np.ndarray) -> np.ndarray:
        observation = self.build_observation(state)
        result = self.policy.infer(observation)
        actions = np.asarray(result["actions"], dtype=np.float32)
        if actions.ndim == 1:
            actions = actions.reshape(1, -1)
        return actions


# 使用示例
camera_map = omnipicker_camera_map(enable_wrist_l=False)
camera_name_map = {k: v[0] for k, v in camera_map.items()}

cameras = bring_up_cameras(camera_map, port_timeout=30.0, frame_timeout=30.0)
target_hw = probe_camera_hw(cameras, camera_map)

runner = OpenPIPolicyRunner(
    host="localhost",
    port=8010,
    prompt="按红色按钮",
    camera_name_map=camera_name_map,
    cameras=cameras,
```

```
    target_hw=target_hw,
)

state = storage.build_state(storage.obs_callback(env))
action_chunk = runner.infer_action_chunk(state)

# 执行 action_chunk ...

close_cameras(cameras)
```

12.10 常见失败模式与修复

现象	原因	处理
`bring_up_cameras` 超时	ORCA 未渲染 / 端口未监听 / 相机未勾选 Recording	检查 ORCA 窗口是否在渲染；`netstat -anp | grep 7080` 确认端口；重新勾选 Recording
首帧一直等不到	WebSocket 在端口就绪前连接，后台线程已退出	`bring_up_cameras` 已内置 TCP 探测与线程重启；仍失败则重启 ORCA 并重新触发推流
图像是全黑的	`begin_save_video` 未调用或调用失败	确认 `STREAM_TRIGGER_PATH` 目录可写；检查 ORCA 推流状态
策略输出维度不匹配	训练/推理图像 key 不一致	确保 `camera_name_map` 与数据集 `features` 中的相机键一致
`policy.infer` 报 WebSocket 超时	策略服务未启动或防火墙拦截	检查 `serve_policy.py` 日志；使用 `ssh -L 8010:localhost:8010 gpu_server` 做隧道
推理画面与训练画面方向/颜色不一致	使用 `bgr24` 而非 `rgb24`，或忘了 transpose	统一用 `format="rgb24"` 并 `np.transpose(rgb, (2, 0, 1))`
控制频率不稳定、视频时长不够	未做 real-time 节流，wall-clock 快于仿真时间	启用 `--sleep` 或在 `env.step` 后按 `real_time_step` 补眠
左腕相机未生效	未传 `--enable_wrist_l` 或场景未配置	需要三路图像时启动脚本加 `--enable_wrist_l`，并确保场景有 `camera_wrist_l_color`

关闭相机时后台线程抛 `InvalidDataError`	PyAV 在 websocket 关闭瞬间收到不完整的 H.264 包	通常无害，主流程已正常结束；如频繁出现，可先 `close_cameras(cameras)` 再 `env.stop_save_video()`
`image_index` 一直不变	取帧间隔内没有新帧到达，或推流卡顿	结合 FPS 和图像内容判断；真实 ORCA 流中 index 应随新帧递增

12.11 验证相机流是否可用

仓库提供了两个验证脚本，分别用于**离线快速验证工具链**和**在线验证真实 ORCA 场景**。

12.11.1 离线验证（无需 ORCA）

`scripts/test_camera_websocket_mock.py` 会在本地 7080/7090 启动 mock WebSocket 服务器，发送一段 H.264 测试流，然后调用 `bring_up_cameras` 和 `capture_frame_with_idx` 进行取帧验证。

```
cd SouthGrid
conda activate orcalab_lerobot
python scripts/test_camera_websocket_mock.py
```

预期输出：

```
[OK] 已连接 2 路相机，分辨率: (480, 640)
[*] 持续取帧 5.0 秒...
...
========== 验证结果 ==========
  cam_head: 总帧数=99, 平均 FPS=19.6, 黑帧比例=0.0% -> OK
  cam_wrist_r: 总帧数=99, 平均 FPS=19.6, 黑帧比例=0.0% -> OK
[OK] 离线相机流验证通过。
```

> 该脚本仅验证 `lerobot_camera` 工具链本身是否能正确连接 WebSocket、解码 H.264、取到非黑帧。它**不验证 ORCA 场景配置**，也不能保证真实场景下端口和分辨率完全一致。

12.11.2 在线验证（需要 ORCA 运行）

`scripts/verify_camera_stream.py` 会真正连接 ORCA，触发 `env.begin_save_video()`，并通过 `bring_up_cameras` 获取真实相机流。

```
cd SouthGrid
conda activate orcalab_lerobot
python scripts/verify_camera_stream.py \
    --task_config src/examples/dataCollection/g1_omnipicker/g1_button.json \
    --orcagym_addr localhost:50051 \
    --duration 10
```

运行前提：

- OrcaLab 已启动并加载带相机的场景（如 `g1_button.json`）。
- 场景中的 Camera 组件已启用 **UseNvEnc + Color Camera + Recording**。
- 相机端口与 `omnipicker_camera_map()` 中一致（头 7090，右腕 7080，左腕 7070）。

预期输出：

```
[5/6] 触发相机推流: env.begin_save_video('/tmp/verify_camera_stream')
[6/6] 连接 WebSocket 相机并等待首帧...
  ✓ 端口 7090 (camera_head_color) 已就绪
  ✓ 端口 7080 (camera_wrist_r_color) 已就绪
[OK] 已连接 2 路相机，分辨率: (480, 640)
[*] 开始持续取帧 10 秒...
...
========== 验证结果 ==========
  cam_head: 总帧数=198, 平均 FPS=19.8, 黑帧比例=0.0% -> OK
  cam_wrist_r: 总帧数=198, 平均 FPS=19.8, 黑帧比例=0.0% -> OK
[OK] 相机推流验证通过。
```

> 如果验证失败，脚本会打印具体检查项：ORCA 是否在渲染、相机组件是否勾选 Recording、端口是否一致等。

12.11.3 只读取头部相机

如果只想快速确认头部相机（head camera）是否能正常取图，可以使用 `scripts/test_head_camera.py`。该脚本不依赖 `DataCollectionManager`，直接通过 `OrcaGymLocalEnv` 连接 ORCA，触发推流并读取头部相机。

```
cd SouthGrid
conda activate orcalab_lerobot
python scripts/test_head_camera.py \
    --orcagym_addr localhost:50051 \
    --duration 10 \
    --save_preview
```

脚本会自动检测本地 7070-7100 范围内哪些端口正在监听。如果默认端口不匹配，可用 `--head_port` 指定实际端口：

```
python scripts/test_head_camera.py --head_port 7070 --duration 10 --save_preview
```

> **注意**：实际端口以 ORCA 场景中 Camera 组件的 `ColorPort` 为准。`omnipicker_camera_map()` 中的默认值（头 7090、右腕 7080、左腕 7070）可能因场景配置不同而变化。如果连接失败，脚本会列出当前检测到的监听端口供参考。

预期输出：

```
[1/3] 连接 ORCA: localhost:50051
[*] 已加载模型，body 数量: 74
[*] 当前本地 7070-7100 监听端口: [7070, 7071]
[2/3] 触发头部相机推流: env.begin_save_video('C:\Users\...\test_head_camera_stream')
[*] 触发推流后本地 7070-7100 监听端口: [7070, 7071]
[3/3] 连接 WebSocket 头部相机 (端口 7070)...
  ✓ 端口 7070 (camera_head_color) 已就绪
[OK] 头部相机已就绪，分辨率: (480, 640)
[*] 开始读取头部相机 10 秒...
  head camera: index=19, shape=(480, 640, 3), max=255, fps=664.5
  ...
========== 头部相机读取结果 ==========
  总帧数=197, 平均 FPS=19.7, 黑帧比例=0.0%
  状态: OK
  预览图已保存: /tmp/test_head_camera_preview.jpg
```

12.11.4 分步实现详解：从 ORCA 配置到代码取帧

本节以头部相机为例，完整讲解如何在 OrcaLab 中启用相机，以及 `test_head_camera.py` 中每一行代码背后的原理。

步骤 1：在 OrcaLab 中启用头部相机推流

1. 在 OrcaLab 场景中选中机器人（如 `g1_omnipicker`）。
2. 展开机器人层级，找到并点击 **`mujoco_wrap`**。
3. 在顶部工具栏中点击 **“递归显示”**（或对应的层级展开图标），确保右侧 Outliner 能显示 `mujoco_wrap` 下的全部子节点。
4. 在右侧属性面板中点击 **“全部显示”**，让隐藏的相机组件也可见。
5. 在层级树中找到需要的相机，例如 **`camera_head`**（或 `camera_head_color`），点击选中。
6. 在右侧出现的相机属性窗口中，按顺序勾选以下三项（顺序很重要）：

- **Color Camera**：启用彩色图像输出；
- **UseNvEnc**：使用 NVIDIA 硬件编码，降低 CPU 占用；
- **IsRecording**：开始推流/录制。

7. 记录 **ColorPort** 的值（例如 `7070`），代码里要用这个端口连接 WebSocket。

> 如果是腕部相机（`camera_left` / `camera_right`），同样操作，只是端口可能不同。最终所有启用了 `IsRecording` 的相机会在 `begin_save_video()` 后被 ORCA 通过 WebSocket 推送到对应端口。

步骤 2：代码整体流程

```
# 1. 建立与 ORCA 的连接，加载当前场景模型
env = OrcaGymLocalEnv(
    frame_skip=5,
    orcagym_addr="localhost:50051",
    agent_names=["g1_omnipicker"],
    time_step=0.002,
)

# 2. 通知 ORCA 开始保存视频/推流
env.begin_save_video("/tmp/test_head_camera_stream")

# 3. 等待并连接相机 WebSocket
from dataStorage.lerobot_camera import bring_up_cameras, capture_frame_with_idx
camera_map = {"camera_head_color": ("cam_head", 7070)}
cameras = bring_up_cameras(camera_map, port_timeout=30.0, frame_timeout=30.0)

# 4. 循环取帧
images, indices = capture_frame_with_idx(cameras, camera_map, target_hw=(480, 640))
head_image = images["cam_head"]   # shape: (480, 640, 3), RGB
head_index = indices["camera_head_color"]
```

步骤 3：`OrcaGymLocalEnv` 连接 ORCA

```
env = OrcaGymLocalEnv(
    frame_skip=5,
    orcagym_addr="localhost:50051",
    agent_names=["g1_omnipicker"],
    time_step=0.002,
)
```

- `orcagym_addr`：OrcaLab gRPC 服务地址，默认 `localhost:50051`。
- `agent_names`：告诉 ORCA 你想控制哪个机器人；即使只是读相机也建议填写，确保加载正确的模型。

- `frame_skip`：每调用一次 `env.step()`，仿真前进 `frame_skip * time_step` 秒。
- 这一步会通过 gRPC 从 OrcaLab 下载当前场景的 XML 模型，并在本地初始化 `OrcaGymModel` 和 `OrcaGymData`。

步骤 4：`env.begin_save_video()` 触发推流

```
env.begin_save_video("/tmp/test_head_camera_stream")
```

`begin_save_video` 内部调用 `OrcaGymLocal.begin_save_video()`，向 OrcaLab 发送 `BeginSaveMp4FileRequest`。

- `file_path`：告诉 ORCA 把录制的 MP4 文件存到哪里；实际录制路径为 `{file_path}/video/{camera_sensor_name}.mp4`。
- 该请求会触发 OrcaLab 对所有已启用 `IsRecording` 的相机打开 WebSocket 推流端口（如 7070/7080/7090）。
- 必须调用 `env.render()` 或 `env.step()` 让仿真继续推进，否则 OrcaLab 不会真正开始发送数据。

在 `test_head_camera.py` 中：

```
env.begin_save_video(STREAM_TRIGGER_PATH)
env.render()            # 推进一帧，让 ORCA 开始推流
```

步骤 5：`bring_up_cameras()` 稳健连接 WebSocket

```
cameras = bring_up_cameras(camera_map, port_timeout=30.0, frame_timeout=30.0)
```

`dataStorage/lerobot_camera.py` 中的 `bring_up_cameras` 做了三件事：

1. **TCP 端口探测**：先扫描指定端口（如 7070）是否已被 OrcaLab 监听；
2. **WebSocket 连接**：端口就绪后，为每个相机创建 `CameraWrapper` 并启动后台线程；
3. **首帧等待**：循环检查每个相机是否收到第一帧，若后台线程异常退出则自动重启（最多 3 次）。

```
from orca_gym.sensor.rgbd_camera import CameraWrapper

cam = CameraWrapper(name="camera_head_color", port=7070)
cam.start()   # 启动子线程，在子线程里跑 asyncio 事件循环连接 ws://localhost:7070
```

`CameraWrapper` 内部核心逻辑（简化版）：

```
async def do_stuff(self):
    uri = f"ws://localhost:{self.port}"
    async with websockets.connect(uri) as websocket:
        rawData = io.BytesIO()
        container = None
        while self.running:
            data = await websocket.recv()
            data = data[8:]                 # 前 8 字节是时间戳，后面是 H.264 视频数据
            rawData.write(data)
            rawData.seek(cur_pos)
            if cur_pos == 0:
                container = av.open(rawData, mode='r')   # PyAV 打开 H.264 流

            for packet in container.demux():
                frames = packet.decode()
                for frame in frames:
                    self.image = frame.to_ndarray(format='bgr24')
                    self.image_index += 1
                    self.received_first_frame = True
            cur_pos += len(data)
```

关键点：

- WebSocket 每条消息的前 8 字节是**小端序时间戳**（单位：纳秒），后面才是 H.264 视频码流；
- `CameraWrapper` 用 `PyAV`（`av` 库）实时解封装 H.264 并解码为 BGR 图像；
- 解码后的最新帧保存在 `self.image`，索引保存在 `self.image_index`；
- `get_frame(format="rgb24")` 会把 BGR 转成 RGB 返回。

步骤 6：`capture_frame_with_idx()` 取帧

```
images, indices = capture_frame_with_idx(cameras, camera_map, target_hw=(480, 640))
head_image = images["cam_head_color"]
head_index = indices["camera_head_color"]
```

`capture_frame_with_idx` 的实现：

```
def capture_frame_with_idx(cameras, camera_map, target_hw):
    H, W = target_hw
    images, indices = {}, {}
    for env_name, (key, _port) in camera_map.items():
        cam = cameras[env_name]
        frame, idx = cam.get_frame(format="rgb24")    # 取最新解码帧（BGR→RGB）
        if frame.shape[0] != H or frame.shape[1] != W:
            frame = cv2.resize(frame, (W, H), interpolation=cv2.INTER_AREA)
        images[key] = np.ascontiguousarray(frame, dtype=np.uint8)
        indices[env_name] = idx
    return images, indices
```

- `env_name`：ORCA 场景中相机的 sensor 名称，如 `camera_head_color`；
- `key`：你自己定义的简称，如 `cam_head`，后续作为字典 key 使用；
- `target_hw`：统一输出分辨率 `(H, W)`；如果相机原始分辨率不同，会用 `cv2.resize` 缩放到目标尺寸；
- `image_index`：每收到一帧新解码图像就加 1；如果连续两次取到的 index 相同，说明没有新帧到达（推流卡顿或帧率不足）。

步骤 7：停止推流与关闭连接

```
close_cameras(cameras)        # 停止 CameraWrapper 后台线程
env.stop_save_video()         # 通知 ORCA 停止推流/录制
env.close()                   # 关闭 gRPC 连接
```

`close_cameras` 只是设置 `cam.running = False`，让 `CameraWrapper` 的后台线程自然退出；`env.stop_save_video()` 通知 OrcaLab 停止录制并写盘；`env.close()` 释放 gRPC 资源。

完整最小示例

下面是可以直接运行的最小示例（假设 ORCA 已启动且头部相机端口为 7070）：

```
import os
import sys
import time
import cv2
import numpy as np

project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
if project_root not in sys.path:
    sys.path.insert(0, project_root)
src_root = os.path.join(project_root, "src")
if src_root not in sys.path:
    sys.path.insert(0, src_root)

from dataStorage.lerobot_camera import (
    bring_up_cameras,
    capture_frame_with_idx,
    close_cameras,
    probe_camera_hw,
)
from orca_gym.environment.orca_gym_local_env import OrcaGymLocalEnv

STREAM_TRIGGER_PATH = os.path.join(os.environ.get("TEMP", "/tmp"),
"head_camera_demo")
CAMERA_MAP = {"camera_head_color": ("cam_head", 7070)}

env = OrcaGymLocalEnv(
    frame_skip=5,
    orcagym_addr="localhost:50051",
    agent_names=["g1_omnipicker"],
    time_step=0.002,
)

os.makedirs(STREAM_TRIGGER_PATH, exist_ok=True)
env.begin_save_video(STREAM_TRIGGER_PATH)
env.render()
time.sleep(1.0)

cameras = bring_up_cameras(CAMERA_MAP, port_timeout=30.0, frame_timeout=30.0)
if not cameras:
    print("相机连接失败")
    sys.exit(1)

target_hw = probe_camera_hw(cameras, CAMERA_MAP)
```

```
print(f"相机分辨率: {target_hw}")

for i in range(100):
    images, indices = capture_frame_with_idx(cameras, CAMERA_MAP, target_hw)
    frame = images["cam_head"]
    idx = indices["camera_head_color"]
    print(f"frame {i}: index={idx}, shape={frame.shape}, max={frame.max()}")
    time.sleep(0.05)

cv2.imwrite("head_camera_preview.jpg", cv2.cvtColor(frame, cv2.COLOR_RGB2BGR))

close_cameras(cameras)
env.stop_save_video()
env.close()
```

第 13 章

独立机器人 XML 模型与跨平台训练

`SouthGrid` 仓库在 `src/model/` 目录下提供了两个可直接加载的机器人 XML 模型：

```
src/model/
├── G1_omnipicker/
│   ├── g1_omnipicker.xml          # 智元 G1 + OmniPicker 夹爪
│   └── assets/                    # meshes / textures
└── G1_Pick/
    ├── g1_pick_southgrid.xml      # 宇树 G1 + OmniPicker 夹爪
    └── assets/
```

本章说明这两个模型的结构、如何在自己的仿真平台或真机上训练，以及**最终如何回到 ORCA 平台进行自动计分**。

13.1 模型定位：参考实现，不是唯一入口

这两个 XML 模型有双重用途：

1. **作为 ORCA 场景内的机器人资产配置参考**：ORCA Studio 加载场景时，会引用与这些 XML 结构一致的 body/joint/site/camera 名称。开发者在 ORCA 内做数据采集和推理时，通常不需要直接编辑 XML。
2. **作为自定义平台训练的起点**：如果用户希望在 MuJoCo、Isaac Gym/Sim、Genesis 或其他真机平台上先进行算法验证和预训练，可以直接复用这两个 XML 的拓扑、质量和运动学参数。

> **核心原则**：训练阶段可以在任意平台完成，但最终要参加评分，必须把策略部署到 ORCA 场景中，并通过 `ScorerClient` 上报结果。评分系统只认 ORCA 内的实际执行结果，不认训练平台。

13.2 两个 XML 的机器人结构

13.2.1 `G1_omnipicker/g1_omnipicker.xml`（智元 G1 + OmniPicker）

该模型以 `robot_holder1` 为根，通过 `free_joint` 浮动在世界坐标系中，整体结构如下：

部位	body / joint 命名规律	主要关节	备注

躯干	`body_link1` → `body_link2`	`idx01_body_joint1`、`idx02_body_joint2`（自定义标签）	躯干可俯仰和侧倾
头部	`head_link1` → `head_link2`	`idx11_head_joint1`、`idx12_head_joint2`	头部相机 `camera_head` 挂在 `head_link2` 下
左臂	`arm_l_link1` → `arm_l_end_link`	`idx21_arm_l_joint1` ~ `idx27_arm_l_joint7`	7-DOF 机械臂
右臂	`arm_r_link1` → `arm_r_end_link`	`idx61_arm_r_joint1` ~ `idx67_arm_r_joint7`	7-DOF 机械臂
左手 OmniPicker	`gripper_l_inner_link*` / `gripper_l_outer_link*`	`idx31/32/33/39/41/42/43/49_gripper_l_*`	8 个关节的并联夹爪
右手 OmniPicker	`gripper_r_inner_link*` / `gripper_r_outer_link*`	`idx71/72/73/79/81/82/83/89_gripper_r_*`	8 个关节的并联夹爪
末端 site	`ee_center_site_l` / `ee_center_site_r`	—	位于手掌前方 0.08m 处，用于 IK 和距离计算
腕部相机	`camera_left` / `camera_right`	—	分别挂在左右 `arm_*_end_link` 下
移动底盘	`wheel_*_link`	`wheel_l/r_joint`、四轮转向 `wheel_*_steer_joint`	全向移动平台

关键参数：

- `<compiler angle="radian" />`：所有角度单位为弧度。
- 关节范围示例：`arm_l_joint1` -3.14 ~ 3.14 rad；`arm_l_joint4` -1.75 ~ 0 rad。
- 臂部电机力矩范围：±60 N·m（大关节），±30 N·m（腕部关节）。
- 夹爪电机力矩范围：±10 N·m。
- 视觉分组：group="1" 为可视化 mesh，group="3" 为碰撞体。

13.2.2 `G1_Pick/g1_pick_southgrid.xml`（宇树 G1 + OmniPicker）

该模型以 `pelvis` 为根，使用 `<freejoint name="floating_base_joint"/>` 浮动，整体结构如下：

部位	body / joint 命名规律	主要关节	备注
躯干	`pelvis` → `waist_yaw_link_rev_1_0` → `waist_roll_link_rev_1_0` → `torso_link_rev_1_0`	`waist_yaw_joint`、`waist_roll_joint`、`waist_pitch_joint`	腰部三自由度
头部	头部 mesh 挂在 `torso_link_rev_1_0` 下	—	头部相机 `head_cam` 挂在躯干上
左腿	`left_hip_pitch_link` → … → `left_ankle_roll_link`	`left_hip_pitch/roll/yaw_joint`、`left_knee_joint`、`left_ankle_pitch/roll_joint`	6-DOF 腿
右腿	对称的 `right_*`	对称的 6-DOF 腿	6-DOF 腿
左臂	`left_shoulder_pitch_link` → … → `left_wrist_yaw_link`	`left_shoulder_pitch/roll/yaw_joint`、`left_elbow_joint`、`left_wrist_roll/pitch/yaw_joint`	7-DOF 机械臂
右臂	对称的 `right_*`	对称的 7-DOF 机械臂	7-DOF 机械臂
左手 OmniPicker	通过 `arm_l_end_link` 接入	`idx31/32/33/39/41/42/43/49_gripper_l_*`	8 关节并联夹爪
右手 OmniPicker	通过 `arm_r_end_link` 接入	`idx71/72/73/79/81/82/83/89_gripper_r_*`	8 关节并联夹爪
末端 site	`ee_center_site_l` / `ee_center_site_r`	—	与智元模型位置定义一致
腕部相机	`camera_left` / `camera_right`	—	挂在左右手臂末端

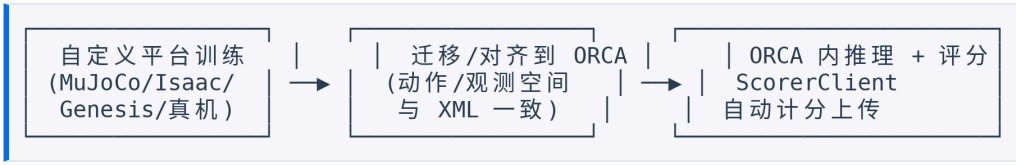
关键参数：

- `<compiler angle="radian" meshdir="assets" autolimits="true" />`：mesh 路径相对于 `assets/`。

- 使用 `<default class="g1">` 统一设置视觉/碰撞分组和材质。
- 肩部关节力矩：±25 N·m；肘部 ±25 N·m；腕部 ±5 N·m。
- 髋/膝/踝关节力矩：±88 ~ ±50 N·m。
- 夹爪结构与智元模型复用同一套 `omnipicker/` mesh。

13.3 自定义平台训练的工作流

如果用户不想直接在 ORCA 里做数据采集，可以按以下流程：



13.3.1 训练时建议保留的关键信息

无论在哪训练，都要保证以下信息与 ORCA 模型一致，否则迁移会很困难：

1. **关节名称与顺序**：`arm_l_joint1` ~ `arm_l_joint7`、`arm_r_joint1` ~ `arm_r_joint7`、夹爪关节名。
2. **动作空间定义**：
   - 推荐以**末端位姿 + 归一化夹爪**作为策略输出（第 12 章的 18 维动作）。
   - 位姿为 `(l_pos_b, l_quat_b, r_pos_b, r_quat_b)`，四元数顺序 `xyzw`。
   - 夹爪为 `[0, 1]` 归一化值，到 ORCA 后再反归一化为电机量程。
3. **观测图像的相机名称和视角**：`camera_head`、`camera_left`、`camera_right` 或对应 `camera_head_color`、`camera_wrist_l_color`、`camera_wrist_r_color`。
4. **末端 site 名称**：`ee_center_site_l` / `ee_center_site_r`，评分系统会读取这些 site 位置计算距离。
5. **机器人根 body 名称**：智元为 `robot_holder1`，宇树为 `pelvis`；稳定控制时需要知道浮动基。

13.3.2 在自定义 MuJoCo 环境中加载 XML

```
import mujoco
import numpy as np

model_path = "src/model/G1_omnipicker/g1_omnipicker.xml"
model = mujoco.MjModel.from_xml_path(model_path)
data = mujoco.MjData(model)

# 查看关节名
for i in range(model.njnt):
    print(i, mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_JOINT, i))

# 查看 body 名
for i in range(model.nbody):
    print(i, mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_BODY, i))

# 查看 site 名（末端位置）
for i in range(model.nsite):
    print(i, mujoco.mj_id2name(model, mujoco.mjtObj.mjOBJ_SITE, i))
```

> 注意：这里用 MuJoCo 仅表示一种可用的刚体求解器；ORCA 支持多物理场后端，MuJoCo 是其中之一。

13.4 回到 ORCA 评分：必须满足的条件

自定义平台训练完成后，要在 ORCA 里自动化计分，必须：

1. **使用 ORCA 提供的场景和机器人资产**：场景 `SouthGrid_Competition_2026`、机器人 `g1_omnipicker` 或 `g1_pick`。自己改过的 XML 除非能在 ORCA 里重新注册并加载，否则评分器不认。
2. **启动一次 attempt**：在推理脚本开始时调用 `scorer_client.start_attempt(task_id=..., task_config=...)`。
3. **执行策略并控制机器人**：通过 `OrcaGymLocalEnv` 的 `env.step(action)` 或 OSC 控制器驱动机器人。
4. **结束 attempt 并上传视频**：任务完成后调用 `scorer_client.finish(...)`，评分服务会自动计算分数并上传视频。

```
from competition_scoring_client import ScorerClient

scorer = ScorerClient(server_url="http://your_scoring_server")
scorer.start_attempt(
    task_id="task_2",
    task_config={"targets": ["red", "green", "blue", "yellow"]}
)

# 运行你的策略 ...
for step in range(max_steps):
    action = policy.infer(obs)
    env.step(action)

scorer.finish(success=True)
```

> 评分规则不对外暴露细节。只要机器人正确按压了按钮或完成了工具归纳动作，`ScorerClient` 会自动上报并计分。

13.5 常见误区

误区	说明
"我在 Isaac Sim 里训练好了，直接提交策略权重就能评分"	不行。评分必须发生在 ORCA 场景中，需要把策略包装成 ORCA 可执行的推理脚本。
"我自己改一个更轻的 XML 来训练，然后直接用到 ORCA"	不建议。ORCA 场景引用的是资产库中的模型，自定义 XML 不会自动生效。
"只要关节名字一样就行，相机位置不重要"	相机位置直接影响策略输入分布，训练时相机视角应尽量与 ORCA 一致。
"夹爪用单一关节控制就够了"	OmniPicker 夹爪有 8 个关节，ORCA 内控制逻辑与训练时的抽象必须一致，否则迁移困难。

13.6 推荐路径

对于大多数参赛者，推荐顺序是：

1. **先在 ORCA 内完成端到端验证**（第 4 ~ 7 章），确认数据采集、推理、评分链路通畅。
2. **如果需要在其他平台预训练**，用 `src/model/` 下的 XML 作为拓扑和质量参考，但保持动作/观测空间与 ORCA 一致。

3. **最终策略必须回到 ORCA 执行**，并通过 `ScorerClient` 上报。

如果确实需要在 ORCA 外做大量训练，建议优先复用 **智元** **`g1_omnipicker`** 模型，因为仓库内的遥操作、数据采集和推理脚本都以它为主力平台；宇树 `g1_pick` 当前只提供遥操作入口和 XML 参考。